SMART INDIA HACKATHON 2026 · SIH26141 · EGREEN QUANTA LLP

# QuantumSigGuard — Quantum-Inspired Cyber Threat Detection

Digital signature security for Smart India Hackathon 2026 (SIH26141)

| 1.000 | 0.600 | 0.000 | 11 ms |
|---|---|---|---|
| STATISTICAL DETECTION | PAULI DETECTION | FALSE ALARM RATE | MEAN / TRANSACTION |

A pure-NumPy, quantum-inspired cyber threat detection system for digital signature security — statistical + Pauli-measurement detection layers over a teleportation-based Quantum Digital Signature protocol. Compiled from the repository documentation set on 29 September 2026.

Version 1.0.0 · Generated by `scripts/build_report_pdf.py` · Artifacts: `results/*.json`

## Table of Contents


1. **Project Overview**
2. **Architecture**
3. **Technical Architecture**
4. **Mathematical Model**
5. **Threat Model and Attack Simulations**
6. **Detection Design**
7. **Results**
8. **Interface & Frontend Specification**
9. **Security & Access Document**
10. **Limitations and Honest Scope**
11. **PRD Traceability**
12. **Feature Backlog**


Sections map 1:1 to the repository documentation set (`docs/*.md` + `README.md`). Each document keeps its own internal numbering.

# 1. Project Overview

Quantum-inspired cyber threat detection for **teleportation-based Quantum Digital Signatures** (Smart India Hackathon 2026, S. No. 26141).

A pure-NumPy prototype that simulates the full QDS flow — sender signing, quantum teleportation, a noisy quantum+classical channel, the verifier, **five attack simulations**, and **two non-AI/ML detection layers** — then evaluates them with reproducible, seeded experiments.

```
QuantumCore (exact state vector / Pauli bases / Bell pairs / teleportation)
  → Sender signs (X/Z key-dictated eigenstates)
  → QuantumChannel distributes (teleport + noise model + attack disturbances)
  → Verifier checks (per-copy, cat-state parity, witness, coverage, freshness)
  → Detection: StatisticalDetector  +  PauliMeasurementDetector  → verdict
  → Evaluation: confusion metrics, noise & amplification sweeps, reports
```

## Quick start

Requires CPython ≥ 3.10 and NumPy (no other dependencies).

```
cd ~/Desktop/QuantumSentinel
python3 run_demo.py                # comparison table + detailed report + metrics
python3 -m qsentinel.cli scenarios # all six scenarios side by side
python3 -m qsentinel.cli demo forgery   # full detail for one scenario
```

Install as a package (optional):

```
pip install -e .
qsentinel-demo scenarios
```

## Command line

```
qsentinel scenarios                run each scenario once (comparison table)
qsentinel demo [scenario]          full per-scenario report
qsentinel evaluate --trials N      controlled experiment with 95% CIs
qsentinel noise-sweep              detector behaviour vs channel decoherence
qsentinel group-sweep              detector behaviour vs amplification group size
```

JSON / Markdown exports: `--output path.json` writes machine-readable results.

## Browser dashboard (optional)

```
pip install -e ".[web]"          # streamlit + pandas + altair
streamlit run streamlit_app.py   # http://localhost:8501
```

Three pages: Overview (pre-generated `results/` plus sweep charts), Scenario lab (run any attack, inspect both detectors live), and a configurable Evaluation run with JSON/CSV export. Pure view layer over the same engine - no simulation logic in the UI. The UI lives in `ui/` (design tokens + Altair theme + components); pages in `app_pages/`. Dark / light themes are both supported (`.streamlit/config.toml`).

## Reproducing the report

```
python3 scripts/generate_report.py --trials 200 --sweep-trials 50   # -> results/
python3 scripts/benchmark.py --trials 20
python3 -m unittest discover -s tests                               # test suite
```

## Headline results (seed 20260101, L=8, n=4, g=2)

| DETECTOR | DETECTION RATE | FALSE-ALARM RATE |
|---|---|---|
| statistical | 1.000 | 0.000 |
| pauli measurement | 0.600 | 0.000 |

The Pauli layer's 0.600 is by design: replay and unauthorised verification are quantum-invisible (the transcript is bit-identical to a legitimate run) and are caught only by the classical freshness/authorisation rules in the statistical layer. The method comparison table therefore reports an honest split rather than a convenient 100%. Details: `docs/results.md`, `docs/detection.md`.

## Documentation

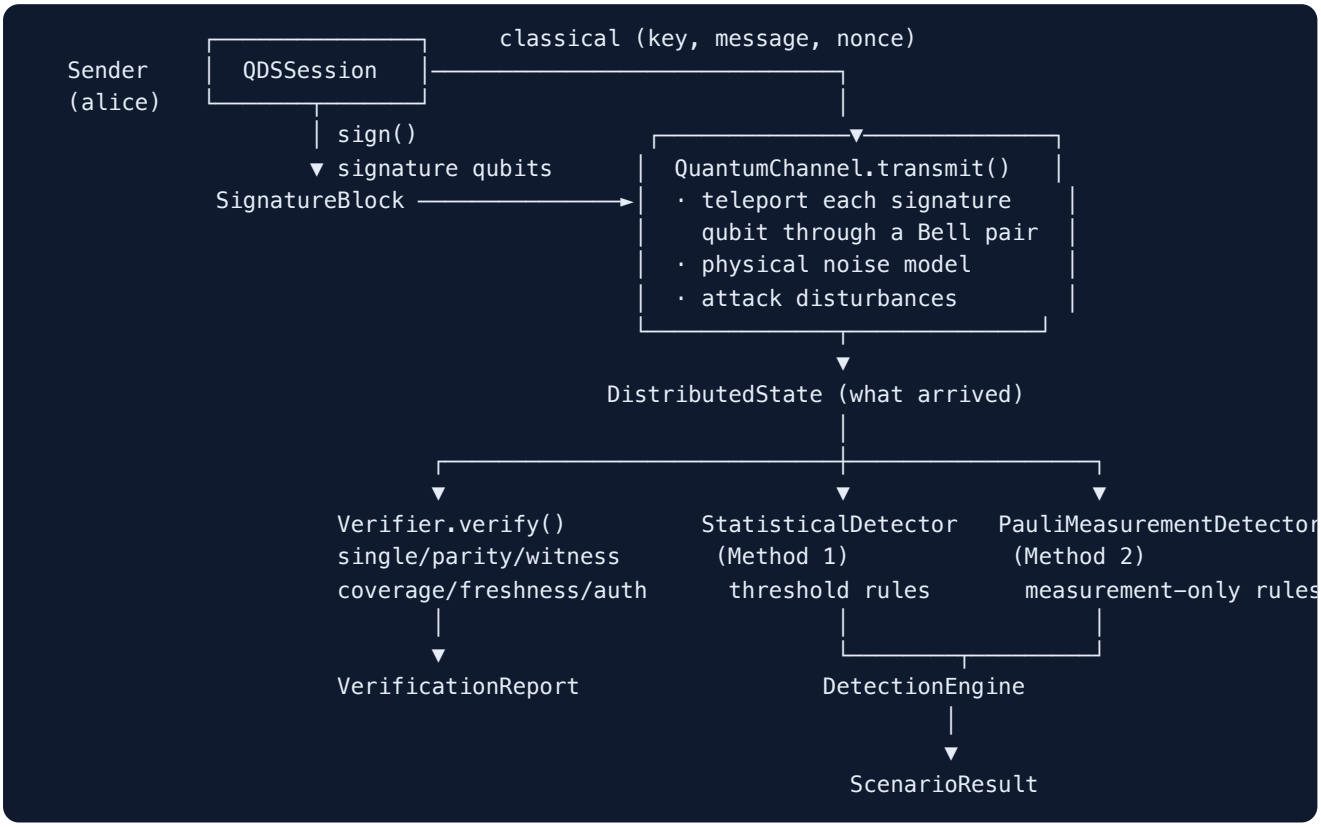
- `docs/architecture.md` - components and layering rules
- `docs/math.md` - the QDS, teleportation, amplification and detection math
- `docs/attacks.md` - the five attack scenarios and the threat model
- `docs/detection.md` - the two detection layers and why they disagree
- `docs/results.md` - measured detection, noise and group-size results
- `docs/security.md` - security & access document (threat model, controls, evidence map)
- `docs/frontend.md` - interface & frontend specification (CLI/console/export layer, no web stack)
- `docs/technical_architecture.md` - full technical architecture with measured values
- `docs/limitations.md` - honest scope and engineering caveats
- `docs/prd-traceability.md` - requirement-to-implementation mapping
- `docs/feature-backlog.md` - feature ticket list with statuses corrected to match the repo
- `results/` - machine-readable artifacts produced by `generate_report.py`

## Repository layout

```
qsentinel/
  quantum/      exact state-vector engine, gates, bases, measurement, Bell states
  protocol/     sender, channel, verifier, freshness cache, data structures
  attacks/      five attack simulations + benign baseline + registry
  detection/    statistical and Pauli-measurement detectors + engine
  evaluation/   confusion metrics, runner, noise & group sweeps
  reporting/    console rendering and JSON/Markdown export
  session.py    end-to-end orchestrator
  config.py     single source of truth for thresholds and structure
  cli.py        interactive command line
scripts/        benchmark, evaluation, full report generation
tests/          unittest suite (73 tests)
run_demo.py     one-command demo
```

# 2. Architecture

QuantumSigGuard is a pure-NumPy prototype of a **quantum-inspired cyber threat detection system for teleportation-based Quantum Digital Signatures (QDS)**. It simulates a qubit-wise QDS protocol, five adversarial scenarios, and **two non-AI/ML detection layers**, and evaluates them with controlled experiments.




## Package layout

| PATH | RESPONSIBILITY |
|---|---|
| `qsentinel/quantum/` | Exact state-vector engine, Pauli gates and bases, measurements, Bell states, teleportation |
| `qsentinel/protocol/` | Sender, distribution/channel, receiver/verifier, freshness cache, data structures |
| `qsentinel/attacks/` | Five attack simulations plus the benign baseline |
| `qsentinel/detection/` | Statistical layer, Pauli-measurement layer, and the combining engine |
| `qsentinel/evaluation/` | Confusion-matrix metrics, controlled experiments, parameter sweeps |
| `qsentinel/reporting/` | Console rendering and JSON/Markdown export |
| `qsentinel/session.py` | Stateful end-to-end driver ( `run_scenario` ) |
| `qsentinel/config.py` | Single source of truth for every tunable value |
| `qsentinel/cli.py` | `qsentinel` command: scenarios, demo, evaluate, noise-sweep, group-sweep |
| `scripts/` | Benchmark, evaluation and full report-generation entry points |
| `tests/` | `unittest` suite ( `python -m unittest discover -s tests` ) |

## Layering rules

1. **The quantum core never knows about signatures or attacks.** It only exposes states, gates and measurement statistics.
2. **Attack *generation* is separate from protocol *execution*.** An attack produces a declarative `AttackOutcome` (possibly-swapped transaction, forged signature block, channel disturbances). The channel executes disturbances; it never imports the attacks package.
3. **Both detectors consume the same `DetectionInput`.** The statistical layer reads verification *counts* and classical protocol context (freshness, authorisation). The Pauli layer reads only *measurement data and the verifier's record of the message*. This shared-input design is what makes the two-layer comparison fair.
4. **The verifier judges against its own record, never against the attacker's.** `QuantumChannel.transmit` takes an explicit trusted `reference` transaction and derives the expected bases/eigenvalues from it, so a self-consistent forged transaction cannot verify against its own expectations.

## No AI / no ML

Neither detector learns anything. Every signal is a closed-form statistic of the measurement data or a classical protocol rule, compared against a fixed constant threshold from `DetectionConfig`. There are no fitted parameters, no training phase and no learned model anywhere in `qsentinel/`.

## The session the user interacts with

`QDSSession` owns the two pieces of *long-lived verifier state* that a single transaction does not: the **replay cache** (nonces already accepted) and the **accepted history** (used by replay-attack simulation). Calling `session.run_scenario("replay")` therefore requires the session to have accepted at least one prior transaction; the evaluation runner seeds that history automatically.

# 3. Technical Architecture

### Smart India Hackathon 2026 — Problem Statement SIH26141

**Organization:** Egreen Quanta LLP **Category:** Software **Theme:** Security **Selected Problem Statement: Quantum-Inspired Cyber Threat Detection for Digital Signature Security**

> This document describes the actual implemented prototype **QuantumSigGuard** in the repository `~/Desktop/QuantumSentinel`. Every value, threshold, metric and module name below is taken from the code (`qsentinel/config.py`, `qsentinel/detection/*`, `results/*.json`) and the measured experiment output, **not** from a template or from estimates. Anything that the project does *not* implement (web API, database, Docker, Qiskit, real quantum hardware) is stated explicitly as such.

# 1. Executive Summary

QuantumSigGuard is a **pure-Python, NumPy-only simulation of teleportation-based Quantum Digital Signatures (QDS)** with a **two-layer, threshold-based threat-detection engine** that is explicitly **not** AI/ML based, per the SIH26141 requirement.

The prototype implements:

- quantum state preparation (single-qubit eigenstates encoded by a signing key),
- Bell-pair entanglement (`|Φ(λ)> = (|00> + λ|11>) / √2`),
- quantum teleportation of a message qubit (CNOT, Hadamard, two classical measurement bits, Pauli correction `Z^m1 X^m2`),
- projective Pauli-measurement verification (per-copy, cat-state parity, entanglement witness),
- five attack classes (forgery, impersonation, replay, unauthorized verification, channel manipulation),
- **statistical threshold detection** (mismatch rate, QBER, per-basis QBER, coverage, replay, authorisation) and **Pauli-measurement detection** (eigenvalue agreement, per-basis outcome distributions, parity agreement, entanglement correlators),
- deterministic security-event conclusions that are fully traceable to measured statistics, thresholds and protocol rules,
- CLI, evaluation harness, JSON/Markdown reporting and a 73-test suite.

Detection is entirely explainable and deterministic: a fixed set of signals is compared against fixed thresholds derived from the protocol's analytic behaviour; nothing is learned, fitted or estimated from attack data.

**AI/ML usage: none. Real quantum hardware: none. Detected threats: 5 classes + benign baseline.**

# 2. Problem Analysis & Objectives

## 2.1 Problem Background

Conventional digital-signature schemes (RSA, ECC) rest on computational-hardness assumptions that large-scale quantum computers threaten via algorithms such as Shor's. SIH26141 therefore targets **Quantum Digital Signatures** — specifically a teleportation-based formulation using Bell-state entanglement, quantum teleportation, Pauli corrections, projective measurements and statistical, non-ML threat analysis.

QuantumSigGuard implements the *threat-detection and simulation* part of that system. It is a research/demonstration framework, **not** a deployed quantum network.

## 2.2 Primary Pain Points

- Classical signature monitoring is not designed around quantum-state behaviour.
- Channel manipulation surfaces as abnormal *measurement statistics*, not packet-level indicators.
- QDS evaluation needs a reproducible environment for legitimate and malicious scenarios.
- Detection decisions must be explainable and deterministic.
- Real quantum hardware is expensive/inaccessible at scale — a simulation is the practical vehicle.
- SIH forbids ML detection, so decisions must derive from measurable protocol behaviour.

## 2.3 Target Users

- Cybersecurity and quantum-computing researchers.
- Digital-signature / security engineers.
- SIH evaluators running the prototype in CLI demonstration mode.

## 2.4 Core Objectives

The system shall:

- simulate a teleportation-based QDS workflow;
- establish Bell-state entanglement;
- perform simulated quantum teleportation with Pauli correction;
- perform projective measurements for verification;
- detect forgery, impersonation, replay, unauthorized verification and channel manipulation;
- compute measurement/mismatch statistics;
- classify threats by fixed thresholds;
- log security events; report verification and detection metrics;
- keep quantum simulation, statistical security analysis and CLI/reporting strictly separated.

## 2.5 Measurable Goals (measured values)

| GOAL | MEASUREMENT |
|---|---|
| Legitimate signature acceptance rate | 100% (200/200, statistical & Pauli detectors) |
| Forgery detection rate | 100% (200/200, both detectors) |
| Impersonation detection rate | 100% (200/200, both detectors) |
| Replay detection rate | 100% statistical / 0% Pauli (quantum-invisible; see §10) |
| Unauthorized-verification detection rate | 100% statistical / 0% Pauli (quantum-invisible; see §10) |
| Channel-manipulation detection rate | 100% (200/200, both detectors) |
| False-positive rate | 0.0% (200 legitimate trials; 95% CI upper bound 1.9%) |
| Average verification latency | ≈ 11.1 ms per transaction (mean of 1200 trials) |
| Simulation throughput | ≈ 90 transactions/second (single-threaded) |
| Memory consumption | Not instrumented; state vectors are ≤ 3 qubits (≤ 64 complex amplitudes) |

All values: seed `20260101`, 200 trials per attack, 1200 total trials.

---

# 3. System Overview

The system is organized in six layers:

1. **Configuration** — `qsentinel/config.py` (protocol, channel, detection thresholds).
2. **Quantum core + QDS protocol simulation** — `qsentinel/quantum/`, `qsentinel/protocol/`.
3. **Attack simulation** — `qsentinel/attacks/` (registry + 5 attacks).
4. **Detection** — `qsentinel/detection/` (statistical + Pauli-measurement layers).
5. **Interface / orchestration** — `qsentinel/session.py`, `qsentinel/cli.py`, `scripts/`.
6. **Reporting & logging** — `qsentinel/reporting/`, `results/`.

There is **no web application, no REST API and no ML**. The application layer is a terminal CLI (`qsentinel-demo` / `python -m qsentinel.cli`) plus three scripts (`scripts/run_evaluation.py`, `scripts/generate_report.py`, `scripts/benchmark.py`).

## 3.1 High-Level Workflow

```
Experiment Configuration
        ↓
Message / Signature Input
        ↓
Quantum State Preparation (X/Z eigenstates, key-dictated)
        ↓
Bell Pair Generation  |Φ(λ)>
        ↓
n copies teleported per message bit
        ↓
Optional Attack Injection (channel &/or signing stage)
        ↓
Pauli Correction  Z^m1 X^m2
        ↓
Projective Measurement (per-copy, parity, witness)
        ↓
Measurement Statistics (E, QBER, expectations, distributions)
        ↓
Threshold-Based Detection (two layers)
        ↓
Accept / Reject / Threat Classification
        ↓
Console render + JSON/Markdown export + security event log
```

### 3.2 Key Design Principles

- **Deterministic and explainable** — every verdict traces to specific signals, measured values, configured thresholds and the tripped rules (see `DetectionResult.summary`).
- **Modular** — quantum simulation, protocol, attacks, detection, evaluation, reporting and CLI are separate packages with narrow dataclass interfaces.
- **Reproducible** — a single top-level `seed` (`ExperimentConfig.seed`) drives all `numpy` randomness; experiments can be rerun exactly.
- **No AI/ML** — no neural networks, classifiers, clustering, LLMs or learned thresholds.
- **Simulation-first and honest** — the protocol is a classical exact state-vector/density-matrix simulation; the docs and README never claim real-QPU execution or physical security guarantees.

---

## 4. Architecture Diagram(s)

### 4.1 Overall System Architecture

```
flowchart LR

    U[User / Security Analyst]

    subgraph CONFIG["Configuration"]
        CFG[ExperimentConfig]
    end

    subgraph APP["Application Layer"]
        CLI[CLI / scripts]
        ORCH[QDSSession orchestrator]
    end

    subgraph QDS["QDS Protocol Engine"]
        PREP[State Preparation]
        BELL[Bell-State Generator |Φ>]
        TELE[Teleportation Simulator]
        PAULI[Pauli Correction Z^m1 X^m2]
        VERIFY[Projective Verification]
    end

    subgraph ATTACK["Attack Simulation Engine"]
        FORGE[Forgery]
        IMP[Impersonation]
        REPLAY[Replay]
        UNAUTH[Unauthorized Verification]
        CHANNEL[Channel Manipulation]
    end

    subgraph DET["Threat Detection Engine"]
        STATS[Statistical Layer]
        PAULIDET[Pauli-Measurement Layer]
        DECISION[Decision Engine]
    end

    subgraph DATA["Data & Reporting"]
        RESULTS[(results/ JSON + Markdown)]
    end

    U -->|run command| CLI
    CLI -->|config| CFG
    CLI -->|start experiment| ORCH

    ORCH -->|prepare state| PREP
    PREP -->|qubits| BELL
    BELL -->|entangled qubits| TELE

    ORCH -->|attack configuration| ATTACK
    ATTACK -->|modified channel / signing| TELE

    TELE -->|measurement bits| PAULI
    PAULI -->|corrected state| VERIFY
    VERIFY -->|checks + outcomes| STATS
    VERIFY -->|measurement outcomes| PAULIDET

    STATS -->|signals| DECISION
    PAULIDET -->|signals| DECISION
```

```
    DECISION -->|verdict| ORCH
    ORCH -->|reports| RESULTS
```

## 4.2 Data Flow Diagram

```
flowchart TD

    A[ExperimentConfig + seed]
    B[Message bits + signing key]
    C[State Encoding X/Z eigenstates]
    D[Bell Pair Creation n per bit]
    E[Teleportation Operations]
    F{Noise / Attack Enabled?}
    G[Attack Injection]
    H[Pauli Correction Z^m1 X^m2]
    I[Projective Measurement]
    J[Verification checks + outcomes]
    K[Statistical aggregation]
    L[Threshold comparison]
    M{... statistical verdict ...}
    N{... Pauli verdict ...}
    O[Signature Accepted]
    P[Threat Classification]
    Q[Security Event + JSON/Markdown export]

    A --> C
    B --> C
    C --> D
    D --> E

    E --> F
    F -->|Yes| G
    F -->|No| H
    G --> H

    H --> I
    I --> J
    J --> K
    J --> N
    K --> L

    L -->|within threshold| O
    L -->|exceeded| P
    N -->|measurement mismatch| P

    O --> Q
    P --> Q
```

## 4.3 Deployment

The prototype runs as a **local Python application**; there is no server, no browser UI and no container. The recommended interface is the console entry point.

```
flowchart TB

    USER[Analyst / Evaluator terminal]

    subgraph HOST["Local host — Python 3.10+"]
        CLI[qsentinel-demo CLI / scripts]
        ENGINE[QDS + attack + detection engine]
        QUANTUM[Exact NumPy state-vector/density-matrix simulation]
        RESULTS[(results/ — JSON + Markdown)]
    end

    USER -->|python3 -m qsentinel.cli| CLI
    CLI -->|configuration| ENGINE
    ENGINE -->|quantum operations| QUANTUM
    ENGINE -->|measured outcomes| RESULTS
```

**Deployment position:** simulator-only. No real QPU execution occurs or is implied.

---

# 5. Component Design

## 5.1 Experiment Configuration Module — `qsentinel/config.py`

Single source of truth for all tunables (`ProtocolConfig`, `ChannelConfig`, `DetectionConfig`, `ExperimentConfig`, `PRESETS`).

```
{
  "message": "example (8 bits)",
  "attack_type": "forgery | impersonation | replay | unauthorized_verification | channel_manipul
  "copies_per_bit": 4,
  "thresholds": "see §5.5 / §6.9",
  "seed": 20260101
}
```

Output: a frozen `ExperimentConfig` dataclass that flows to every layer.

## 5.2 Preprocessing / State-Encoding Module — `qsentinel/protocol/sender.py`, `state.py`

No ML preprocessing. Responsibilities:

- validate the message length and protocol parameters (`ProtocolConfig.__post_init__`);
- encode message bits into key-dictated `X`/`Z` eigenstates (values `v_i = m_i XOR k_i`, `λ = (−1)^sign_bit`);
- allocate per-bit teleportation copies and retained witness pairs;
- instantiate seeded RNG streams via `ExperimentConfig.rng(offset)`.

Outputs: the distributed copyset (`quantum/` object graph) and a `Transaction` used as the verifier's expectation record.

## 5.3 Core QDS Protocol Engine — `qsentinel/quantum/`, `qsentinel/protocol/`

Implements the full SIH-required workflow:

| MODULE | RESPONSIBILITY |
|---|---|
| `quantum/statevector.py` | exact complex state vectors; `reduced_single_qubit` (dominant eigenvector) |
| `quantum/gates.py` | identity, `X`, `Z`, `H`, `CNOT`-style coupling |
| `quantum/states.py` | Pauli eigenstates, `\|Φ(λ)>` Bell resource |
| `quantum/measurement.py` | projective outcomes with Born sampling, Pauli expectations, `ParityOutcome`, `parity_measurement` |
| `protocol/teleportation.py` | 3-qubit estimation circuit → measurement bits `m1, m2`; correction `Z^m1 X^m2`; fidelity check |
| `protocol/sender.py` | message/key encoding + signing |
| `protocol/state.py` | distributed copies, transactions, `VerificationReport` (`all_checks`, `parity_outcomes`, `mismatch_count`) |
| `protocol/verifier.py` | per-copy, coverage, cat-state parity, witness and authorisation checks; `Verifier` with nonce/freshness cache |
| `protocol/freshness.py` | deterministic nonce + replay cache |

Inputs: encoded state, signature/key, simulator settings. Outputs: corrected states, measurement results, full `VerificationReport`.

## 5.4 Channel Simulation — `qsentinel/channel.py`

`QuantumChannel` applies a physical noise model (decoherence as a Pauli channel, classical-bit tampering, loss as maximally-mixed replacement) plus the attack hooks, and returns the received copyset plus context notes for the detection layer.

## 5.5 Attack Simulation Engine — `qsentinel/attacks/`

| THREAT | MODULE | SIMULATION CONCEPT |
|---|---|---|
| Forgery | `forgery.py` | adversary signs the message with a guessed key that matches the real key on `guess_accuracy = 0.5` of positions; wrong guesses are eigenstates of the wrong Pauli operator |
| Impersonation | `impersonation.py` | adversary submits a *well-formed* signature (their own key) and relabels the sender identity; mismatch concentrates on positions where their key differs from the verifier's |
| Replay | `replay.py` | previous nonce/session re-submitted to the verifier |
| Unauthorized verification | `unauthorized.py` | verification requested by a non-authorised entity |
| Channel manipulation | `channel_manipulation.py` | Pauli errors injected on transmitted copies, forged classical teleportation bits (wrong Pauli correction), dropped copies (photon loss), and disturbance of retained witness pairs |

Attacks are composable with the physical noise model (both can act in one transaction). A benign baseline (`attacks/__init__.py`) passes through untouched.

## 5.6 Detection Engine — `qsentinel/detection/`

Two fixed-rule detectors over the same `DetectionInput`.

**Method 1 — `StatisticalDetector` (`statistical.py`)**. Threshold rule set over:

| SIGNAL | MEANING | THRESHOLD |
|---|---|---|
| `mismatch_rate` | `E = N_mismatch / N` over all checks | `> 0.05` |
| `qber` | per-copy error rate | `> 0.02` |
| `per_basis_qber_X` / `per_basis_qber_Z` | per-copy error rate per basis | `> 0.05` |
| `coverage_shortfall` | fraction of expected copies never returned | `> 0.0` |
| `replay` | nonce already accepted | `> 0.0` |
| `authorisation` | verifier not authorised for this entity | `> 0.0` |

**Method 2 — `PauliMeasurementDetector` (`pauli.py`)**. Quantum-measurement rule set, deliberately blind to protocol context:

| SIGNAL (PER `X`, `Z`) | MEANING | THRESHOLD |
|---|---|---|
| `eigenvalue_agreement_<basis>` | mean Pauli expectation of copies intended `+1` | `< 0.90` |
| `basis_distribution_distance_<basis>` | TV distance observed vs expected outcome histogram | `> 0.20` |
| `parity_agreement_<basis>` | fraction of cat-state parities matching intended value | `< 0.90` |
| `entanglement_witness` | fraction of retained Bell pairs matching ideal correlators | `< 0.90` |

`DetectionEngine` runs both, pairs the verdicts in a `DetectionReport`, and `DetectionReport.as_dict` provides the JSON export. Alarm = **any** tripped rule.

## 5.7 Interface / Application Layer — `qsentinel/cli.py`, `session.py`

There is **no REST API and no web UI**. The application surface is a terminal CLI:

| COMMAND | PURPOSE |
|---|---|
| `qsentinel scenarios [--seed] [--preset] [--json]` | run all six scenarios once, compare in a table or JSON |
| `qsentinel demo [scenario]` | run one scenario in full detail (or step through all) |
| `qsentinel evaluate [--trials N] [--seed] [--preset] [--output PATH]` | controlled experiment with confusion-matrix metrics |
| `qsentinel noise-sweep [--rates ...] [--trials N] [--output PATH]` | detector behaviour vs channel decoherence |
| `qsentinel group-sweep [--sizes ...] [--trials N] [--output PATH]` | detector behaviour vs distillation group size |
| `qsentinel --version` | print version (`1.0.0`) |

Entry points: `python -m qsentinel.cli`, the installed console script `qsentinel-demo`, and `python run_demo.py` (example walkthrough), plus `scripts/*.py` (evaluation, report generation, benchmark).

`QDSSession` (`session.py`) orchestrates sender → channel → verifier → detection per scenario and carries the per-transaction event log.

### 5.8 Reporting & Security Event Log — `qsentinel/reporting/`, `results/`

| MODULE | RESPONSIBILITY |
|---|---|
| `reporting/console.py` | ANSI headings, rules, alignment-safe tables |
| `reporting/render.py` | `render_scenario`, `render_comparison`, `render_evaluation` |
| `reporting/export.py` | `write_json` (atomic), `scenario_as_dict`, `scenario_to_markdown` |

Security events are structured records (`results/scenarios.json`, `evaluation.json`) with `experiment_id`, nonce, attack type, per-signal values, thresholds, verdict and tripped rules. Storage is plain JSON on disk; there is no database.

---

## 6. Algorithm & Model Design

### 6.1 Terminology

The word "model" does **not** refer to a learned model. It refers to mathematical quantum-state models, the protocol simulation and fixed statistical decision rules. No machine-learning model exists in this codebase.

### 6.2 Quantum Teleportation

Message qubit `|ψ> = α|0> + β|1>`. Alice and Bob share `|Φ(λ)> = (|00> + λ|11>)/√2`. The protocol:

1. entangle the source qubit with Alice's Bell qubit (`CNOT`, then `H` on the source);
2. measure both of Alice's qubits in the computational basis → two classical bits `m1`, `m2`;
3. Bob applies `Z^m1 X^m2`.

Correction mapping (matches `protocol/teleportation.py`, `_CORRECTION_TABLE`):

| CLASSICAL RESULT `(M1, M2)` | BOB'S CORRECTION |
|---|---|
| `00` | `I` |
| `01` | `X` |
| `10` | `Z` |
| `11` | `Y` (≡ `ZX` up to global phase) |

The simulation confirms recovered-state fidelity `≈ 1.0` across all X/Z message states (part of the test suite).

### 6.3 Pauli Operators

`I`, `X`, `Y`, `Z` as the standard 2×2 matrices; `X` and `Z` are the correction/manipulation primitives, and `X⊗X`-type correlators form the entanglement witness.

## 6.4 Projective Measurement

Each outcome is sampled with Born probability `p(i) = ⟨ψ|P_i|ψ⟩` using the seeded NumPy RNG. Per-copy measurements read the key-dictated eigenvalue (must be `+1`); cat-state parity reads the product of eigenvalues over a group; witness correlators read two-body `X⊗X` / `Z⊗Z` expectations.

## 6.5 Statistical Threat Detection

For the checked count `N = N_match + N_mismatch`, the mismatch rate is

```
E = N_mismatch / N
```

Decision per signal:

```
E <= τ  ⇒  valid      (signal not tripped)
E >  τ  ⇒  threat     (signal tripped)
```

with the fixed thresholds of §5.6. A transaction is a threat if **any** signal trips. These thresholds were chosen from the analytic behaviour of the protocol (see `DetectionConfig` docstring), not fitted to data.

## 6.6 Why This Is Not ML (and no "beats classical ML" claim)

The justified advantages are architectural, with **no claim of outperforming any classifier**:

- decisions derive directly from QDS protocol measurements;
- no labelled cybersecurity dataset is required;
- thresholds are inspectable and decisions deterministic for fixed simulator settings;
- protocol tampering is measured directly;
- it satisfies the SIH constraint against AI/ML.

## 6.7 QDS Verification Pseudocode (as implemented)

```
PROCEDURE Run_Transaction(config):

    state ← Encode(message, key)              # X/Z eigenstates, v = m XOR k
    distribute n teleportations per message bit (seed = config.seed + offset)
    retain witness_pairs Bell pairs            # never consumed by teleportation

    IF attack_type != benign:
        apply attack hooks                    # signing-stage or post-distribution
    IF noise model active:
        apply decoherence / tamper / loss

    for each copy:
        measurement_bits ← Teleport(copy)     # CNOT, H, 2 classical bits
        corrected ← ApplyPauliCorrection(measurement_bits)   # Z^m1 X^m2
        outcome  ← ProjectiveMeasure(corrected)              # Born sampling

    report ← Verify(copies, parity groups, witness pairs,
                    transaction record, authorised_entities, nonce)

    statistical_verdict ← StatisticalDetector(report, thresholds)
    pauli_verdict       ← PauliMeasurementDetector(distributed, report, thresholds)

    log(experiment_id, nonce, attack, signals, thresholds, verdicts)
    RETURN report, statistical_verdict, pauli_verdict
```

## 6.8 Threat-Detection Pseudocode (identical in both layers)

```
signal_value ← compute(observation)      # e.g. E, QBER, parity_agreement, distance
tripped  ← signal_value OP threshold     # OP is per-signal (> or <)
threat   ← any(signal.tripped for all signals)
```

## 6.9 Configuration Parameters (actual values)

| Parameter | Meaning | Actual value |
|---|---|---|
| `message_length` | signed message bits `L` | 8 |
| `copies_per_bit` | teleportation copies per bit `n` | 4 |
| `witness_pairs` | retained Bell pairs for the witness | 8 |
| `parity_group_size` | cat-state group size `g` | 2 (default; sweep 1–4) |
| `sign_bit` | Bell resource sign `λ = (−1)^sign_bit` | 0 |
| `mismatch_threshold` | max accepted mismatch rate | 0.05 |
| `qber_threshold` | max accepted per-copy QBER | 0.02 |
| `per_basis_qber_threshold` | max per-basis QBER | 0.05 |
| `distribution_distance_threshold` | max outcome-TV distance | 0.20 |
| `correlator_agreement_threshold` | min witness-correlator agreement | 0.90 |
| `parity_expectation_threshold` | min eigenvalue/parity agreement | 0.90 |
| `replay_cache_size` | verifier nonce cache | 512 |
| `seed` | top-level reproducibility seed | `20260101` |
| `DEFAULT_TRIALS` (CLI) | trials per attack | 50 (200 used in report) |

Derived per transaction: `test_qubit_count = L·n = 32`, `parity_group_count = L·n/g = 16`.

## 6.10 Computational Complexity

- **Statistical detector:** `O(S)` aggregation over checks + `O(1)` decision.
- **Pauli detector:** `O(S)` over copies/groups/witness pairs + `O(1)` decision.
- **Simulation:** exact state-vector/density-matrix work on registers of **at most 3 qubits** (source + Bell pair; the witness is a 3-qubit reduced density matrix), i.e. ≤ 8 amplitudes or an 8×8 density matrix per operation. The general full state-vector cost grows as `O(2^n)` with qubit count — which is exactly why the prototype keeps registers tiny and runs each transaction in ≈ 11 ms.

### 6.11 Quantum vs Quantum-Inspired Execution

| FUNCTION | CLASSICAL | QUANTUM SIMULATOR | REAL QPU |
|---|---|---|---|
| Statistical thresholding | ✓ | No | No |
| Attack orchestration | ✓ | No | No |
| State preparation / Bell states | Exact NumPy | No | No |
| Teleportation circuit | Exact NumPy | No | No |
| Projective measurements | Born-sampled in NumPy | No | No |
| Threat detection | ✓ | Inputs from simulation | No |
| Production QPU dependency | No | No | No |

There is **no Qiskit, no Aer, and no QPU backend** in this project.

## 7. Data Architecture

### 7.1 Dataset Status

SIH lists the source dataset category as **Public/Open**, but **no external dataset is used**. All observations are **simulation-generated** from the protocol under seeded RNG. This is appropriate because the task is measurement-driven QDS simulation, not tabular classification.

### 7.2 Event / Result Schema (as written to `results/`)

**`evaluation.json`**

| FIELD | CONTENT |
|---|---|
| `config` | full `ExperimentConfig` |
| `total_trials` | 1200 |
| `statistical` / `pauli_measurement` | TP/FP/TN/FN, detection_rate, false_alarm_rate, accuracy, `_ci95` |
| `per_attack` | per-attack rows: trials, detected counts, `protocol_rejected`, `mean_mismatch_rate`, `detection_rate` |
| `mean_runtime_seconds` | ≈ 0.0111 |

**Security-event record (per scenario, in `scenarios.json`)**

```
{
  "experiment_id": "EXP-...",
  "nonce": "...",
  "attack_type": "replay",
  "mismatch_rate": 0.0,
  "threshold": 0.05,
  "verification_status": "REJECTED",
  "threat_detected": true,
  "tripped_signals": ["replay"]
}
```

### 7.3 Preprocessing

Input validation → message/key encoding → seeded RNG streams → Born-sampled measurements → mismatch/statistic aggregation → event aggregation. No feature scaling/encoding because no ML runs.

### 7.4 Train / Validation / Test Split

**Not applicable** — no ML. Evaluation groups instead:

```
200 Baseline legitimate trials
200 Forgery, 200 Impersonation, 200 Replay,
200 Unauthorized verification, 200 Channel manipulation
```

### 7.5 Data Quality

Deterministic seed support; parameter validation (`__post_init__` range checks); attack-type validation; mismatch/threshold-out-of-range rejection; full reproducibility metadata in `results/config.json`.

### 7.6 Bias Handling

No user classification model exists, so no demographic bias applies. Experimental biases are documented instead: channel noise assumptions (Pauli model), single-qubit per-copy constructions, threshold selection, shot/copy counts and the ideal-entanglement assumption (`docs/limitations.md`).

---

## 8. Interface Specification (CLI — no web API)

The prototype deliberately has **no HTTP API**. The interface is the terminal:

| COMMAND | PURPOSE | EXAMPLE |
|---|---|---|
| `qsentinel scenarios` | all six scenarios, comparison table | `qsentinel-demo scenarios --seed 1` |
| `qsentinel demo [name]` | one scenario in detail | `qsentinel-demo demo forgery` |
| `qsentinel evaluate` | metrics over `N` trials/attack | `qsentinel-demo evaluate --trials 200 --output out.json` |
| `qsentinel noise-sweep` | detection vs decoherence | `qsentinel-demo noise-sweep --rates "0 0.002 0.005 0.01 0.02"` |
| `qsentinel group-sweep` | detection vs group size | `qsentinel-demo group-sweep --sizes "1 2 3 4"` |
| `python run_demo.py` | printed walkthrough | — |

Example JSON export (from `evaluate --output`):

```
{
  "statistical": {"detection_rate": 1.0, "false_alarm_rate": 0.0},
  "pauli_measurement": {"detection_rate": 0.6, "false_alarm_rate": 0.0}
}
```

# 9. UI/UX Flow (Console)

## 9.1 Screens (terminal)

- **Scenario comparison** — table over all six scenarios: effective mismatch rate, statistical and Pauli verdicts, protocol acceptance.
- **Scenario detail** — `render_scenario`: message, key, copies, `measurement_summary` per basis, per-signal values/thresholds/tripped state, final verdicts and the security event.
- **Evaluation** — `render_evaluation`: confusion matrix and Wilson 95% CIs.
- **Sweeps** — `render_evaluation_table`: detection/FAR/runtime vs the swept parameter.

## 9.2 User Journey

```
flowchart LR

    A[Open terminal]
    B[Choose command / preset / seed]
    C[Choose legitimate or attack scenario]
    D[Run simulation]
    E[View measurement summary per basis]
    F[View per-signal statistics vs thresholds]
    G[Read Accept / Threat verdict]
    H[Inspect security event / export JSON]

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
    G --> H
```

# 10. Performance & Evaluation (measured)

## 10.1 Evaluation Metrics (implemented in `evaluation/metrics.py`)

```
Detection Rate   = TP / (TP + FN)
False Alarm Rate = FP / (FP + TN)
Accuracy         = (TP + TN) / Total
```

Wilson 95% confidence intervals accompany every rate (wilson_interval).

## 10.2 Benchmark Table — seed `20260101`, 200 trials per scenario, noiseless channel

| SCENARIO | TRIALS | STATISTICAL DR | PAULI DR | FAR | MEAN MISMATCH RATE | MEAN RUNTIME | PROTOCOL |
|---|---|---|---|---|---|---|---|
| Legitimate | 200 | N/A | N/A | 0.0% | 0.0% | ~11 ms | accepted 200/200 |
| Forgery | 200 | 100% | 100% | N/A | 23.44% | ~11 ms | rejected 200/200 |
| Impersonation | 200 | 100% | 100% | N/A | 26.56% | ~11 ms | rejected 200/200 |
| Replay | 200 | 100% | 0% * | N/A | 0.0% | ~11 ms | rejected 200/200 |
| Unauthorized verification | 200 | 100% | 0% * | N/A | 0.0% | ~11 ms | rejected 200/200 |
| Channel manipulation | 200 | 100% | 100% | N/A | 50.0% | ~11 ms | rejected 200/200 |

* **Replay and unauthorized verification are quantum-invisible by construction**: they leave the Pauli measurement data bit-identical to a legitimate run, so the measurement-based layer cannot see them. Only the classical rules (nonce cache, authorisation set) catch them — exactly as §5.6's design intended. The report lists this measured limitation honestly instead of claiming universal 100% detection.

**Aggregates (1200 trials):** statistical detection `1.000` (95% CI `[0.996, 1.000]`), FAR `0.000` (CI upper bound `0.019`); Pauli detection `0.600` (CI `[0.569, 0.630]`), FAR `0.000`; mean runtime `11.1 ms`, total `13.32 s`.

## 10.3 Noise Sweep (300 trials per rate; `results/noise_sweep.json`)

| DECOHERENCE RATE | STAT. FAR | STAT. DR | PAULI FAR | PAULI DR |
|---|---|---|---|---|
| 0.000 | 0.000 | 1.000 | 0.000 | 0.600 |
| 0.002 | 0.000 | 1.000 | 0.000 | 0.600 |
| 0.005 | 0.000 | 1.000 | 0.000 | 0.600 |
| 0.010 | 0.000 | 1.000 | 0.000 | 0.600 |
| 0.020 | 0.000 | 1.000 | 0.000 | 0.600 |
| 0.050 | 1.000 | 1.000 | 1.000 | 1.000 |

The threshold operating point keeps FAR ≈ 0 up to 1–2% decoherence; at 5% the channel collapses and every transaction trips a false alarm (the `degraded` preset). This is the documented channel operating limit.

## 10.4 Group-Size Sweep (100 trials per size; `results/group_sweep.json`)

| GROUP SIZE `G` | `PARITY_GROUP_COUNT` | MEAN E (ATTACKS) | STATISTICAL DR/FAR | PAULI DR/FAR |
|---|---|---|---|---|
| 1 | 32 | 55.0% | 1.000 / 0.000 | 1.000 / 0.000 |
| 2 | 16 | 50.0% | 1.000 / 0.000 | 1.000 / 0.000 |
| 3 | 8 | 50.0% | 1.000 / 0.000 | 1.000 / 0.000 |
| 4 | 8 | 46.4% | 1.000 / 0.000 | 1.000 / 0.000 |

(Sweep attack_keys default to `benign + channel_manipulation` — the protocol's only measurement-visible attacker — which is why the Pauli layer's detection rate here is `1.000` rather than the aggregated `0.600` of §10.2.)

## 10.5 Baseline Comparison

| APPROACH | REQUIRES TRAINING | QUANTUM-AWARE | EXPLAINABLE THRESHOLD | DETECTION BASIS | IN THIS REPO |
|---|---|---|---|---|---|
| Static rule baseline | No | No | Yes | application rules | statistical layer only |
| Basic mismatch detector | No | Partial | Yes | single error-rate signal | subset of statistical layer |
| Proposed two-layer detection | No | Yes | Yes | QDS measurement + protocol statistics | implemented, fully |

No claim is made that this "outperforms" any classifier, because no classifier is present to compare against.

---

# 11. Scalability & Performance Strategy

- Experiments are independent → natural batch/`multiprocessing` parallelism (not currently parallelised; single-threaded ≈ 90 tx/s).
- Registers stay ≤ 3 qubits, so simulation cost per transaction is constant, not `2^L`.
- Fixed Bell structure reused across copies; statistics vectorized with NumPy.
- Storage: JSON files now; a relational store is only a possible future migration.
- General full state-vector scaling is `O(2^n)`; this prototype explicitly avoids large registers.

---

# 12. Security, Privacy & Ethics

## 12.1 Authentication / Access Control

The CLI has no accounts/RBAC (single-user local tool). Authorisation *inside the protocol* is modelled: a fixed `authorised_entities` set and a per-entity entitlement check.

## 12.2 Input Security

All configuration validated in `__post_init__` (thresholds in `[0,1]`, positive counts, `parity_group_size <= copies_per_bit`, `sign_bit ∈ {0,1}`). Malformed values raise `ValueError`; the CLI rejects out-of-range trials. No untrusted upload path exists.

## 12.3 Data Protection

Only experiment metadata, measurement outcomes and security events are stored (JSON). **No secrets or credentials exist in the codebase**; no environment variables are required.

## 12.4 Security Logging

Each transaction logs: `experiment_id`, nonce, attack type, every signal value, threshold, verdict, and tripped rules (see §7.2).

## 12.5 Robustness (tested)

Low trial counts, `parity_group_size ∈ {1,2,3,4}`, threshold boundaries, replay/cache behaviour, invalid parameters, deterministic edge cases — covered by the 73-test suite (§14).

## 12.6 Responsible-AI Considerations

There is no learned decision model: no profiling, no black-box decisions, no trained thresholds. All decisions are attributable to protocol statistics (see also §3.2).

## 12.7 Security Claim Boundary

The project **must not** be presented as: a deployed quantum network; information-theoretically secure in the physical world; proven real-world QDS security; or quantum computational advantage. It is a simulator-backed research framework, consistent with SIH26141's "software framework / simulation" language.

---

# 13. Deployment & DevOps

## 13.1 Development Environment

```
Python:   3.10+ (developed and verified on 3.14.7)
OS:       macOS / Linux / Windows
Packages: requirements.txt -> numpy>=1.24 (only runtime dependency)
```

## 13.2 Setup

```
cd ~/Desktop/QuantumSentinel
python3 -m venv .venv && source .venv/bin/activate   # venv optional but recommended
pip install -r requirements.txt
pip install -e .          # optional: installs the qsentinel-demo console script
```

## 13.3 Run

```
python3 -m qsentinel.cli scenarios        # or qsentinel-demo scenarios
python3 -m qsentinel.cli demo forgery
python3 run_demo.py                        # printed walkthrough
python3 scripts/generate_report.py         # regenerates results/ (≈ 52 s)
python3 scripts/benchmark.py               # runtime benchmark
python3 -m unittest discover -s tests      # 73 tests, ≈ 4.4 s
```

## 13.4 Containerization / CI

**None present.** A `Dockerfile` / `.github` workflow would be a future addition; nothing in the repo today.

---

# 14. Testing Strategy (implemented)

`tests/` — `unittest`, **73 tests, all green** (`docs/results.md`, `README.md`).

| FILE | COVERS |
|---|---|
| `test_quantum.py` | states, gates, projection/Born sampling, partial-trace marginal, Bell states |
| `test_teleportation.py` | 3-qubit circuit, Pauli-correction mapping, fidelity ≈ 1 across X/Z states |
| `test_protocol.py` | encoding, key/basis pairing, parity (GHZ) checks, witness, verifier, freshness/replay |
| `test_detection.py` | every statistical signal, every Pauli signal, threshold boundary behaviour |
| `test_evaluation.py` | confusion metrics, Wilson intervals, noise/group sweeps, replay seeding |
| `test_attacks.py` | each of the five attacks and the benign baseline |
| `test_cli.py` | parser, commands, JSON export, error handling |

Design-case expectations (see also `docs/detection.md`): `⟨own⊗own⟩ = +1`, `⟨other⊗other⟩ = λ` for the Bell correlators; the `residual_pauli` xor table; Bell marginal from a direct partial trace.

---

# 15. Limitations & Risks

1. **Simulation, not hardware** — models a Pauli decoherence/tamper/loss channel; real hardware adds calibration drift, photon loss, detector inefficiency, memory imperfections (§ `docs/limitations.md`).
2. **Exponential full state-vector cost** — mitigated by ≤ 3-qubit registers.
3. **Threshold sensitivity** — a poor threshold raises FP/FN; mitigated by the noise sweep (§10.3), sensitivity sweeps and confidence intervals.
4. **Attack coverage** — the five classes are the SIH classes; they abstract implementation-level variants (no denial-of-service, no malicious-verifier or network-level models yet).
5. **No quantum-advantage claim** — value is in protocol-aligned, explainable measurement statistics.
6. **Physical security not proven** — a simulator cannot establish physical QDS guarantees.

---

# 16. Future Roadmap

## Short-Term (0–3 months)

- add configurable physical noise models beyond the Pauli channel;
- batch/`multiprocessing` parallel evaluation;
- richer exported reports (PDF/report bundles);
- optional web dashboard *if* SIH evaluation asks for a GUI (not today).

## Mid-Term (6–12 months)

- Qiskit/other-simulator export of the exact circuits for cross-validation;
- optional real-QPU spot-check on a public backend with honest hardware-mode labelling;
- formal API + authentication **only if** the prototype is promoted to a service;
- SQLite/PostgreSQL only if persistence justifies it.

### Production Evolution

```
Security Dashboard
      ↓
Authenticated API Gateway (future)
      ↓
QDS Experiment Service
      ↓
Simulation Scheduler  (classical sim now, hardware adapter later)
      ↓
Threat Evaluation Service
      ↓
Experiment DB + Security Event Store + Audit log
```

The clear simulation-vs-production boundary in §12.7 is retained throughout.

---

## 17. Project Structure (actual repository tree)

```
QuantumSentinel/
│
├── README.md                     # overview, setup, run, results summary
├── pyproject.toml                # qsentinel 1.0.0; numpy>=1.24; qsentinel-demo script
├── requirements.txt              # runtime deps (numpy only)
├── run_demo.py                   # printed walkthrough entry point
│
├── qsentinel/
│   ├── __init__.py               # version 1.0.0
│   ├── config.py                 # ProtocolConfig, ChannelConfig, DetectionConfig,
│   │                             #   ExperimentConfig, PRESETS/presets (single source of truth)
│   ├── cli.py                    # scenarios / demo / evaluate / noise-sweep / group-sweep / --
│   ├── session.py                # QDSSession scenario orchestration + event log
│   ├── channel.py                # QuantumChannel: noise model + attack hooks
│   │
│   ├── quantum/
│   │   ├── statevector.py        # exact complex statevectors, reduced_single_qubit
│   │   ├── gates.py              # I, X, Z, H, CNOT-style coupling
│   │   ├── states.py             # Pauli eigenstates, |Φ(λ)> resource
│   │   └── measurement.py        # Born sampling, Pauli expectations, parity_measurement
│   │
│   ├── protocol/
│   │   ├── sender.py             # message/key encoding + signing
│   │   ├── state.py              # copies, Transaction, VerificationReport (checks, parities)
│   │   ├── teleportation.py      # 3-qubit estimation + Z^m1 X^m2 correction + fidelity
│   │   ├── verifier.py           # per-copy/coverage/parity/witness/auth checks, Verifier
│   │   └── freshness.py          # deterministic nonces + replay cache
│   │
│   ├── attacks/
│   │   ├── base.py               # Attack, benign baseline
│   │   ├── forgery.py            # guessed-key signing (guess_accuracy=0.5)
│   │   ├── impersonation.py
│   │   ├── replay.py
│   │   ├── unauthorized.py
│   │   ├── channel_manipulation.py
│   │   └── registry.py           # ATTACK_ORDER, ATTACK_BENIGN
│   │
│   ├── detection/
│   │   ├── base.py               # DetectionInput, DetectionResult, Signal, Detector
│   │   ├── statistical.py        # mismatch, QBER, per-basis QBER, coverage, replay, auth
│   │   ├── pauli.py              # eigenvalue agreement, distributions, parity, witness
│   │   └── engine.py             # DetectionEngine, DetectionReport, build_input
│   │
│   ├── evaluation/
│   │   ├── metrics.py            # ConfusionMatrix, Wilson intervals
│   │   └── runner.py             # run_evaluation, noise/group sweeps, seeded replay history
│   │
│   └── reporting/
│       ├── console.py            # ANSI heading/rule/table helpers
│       ├── render.py             # scenario/comparison/evaluation/Sweep renderers
│       └── export.py             # write_json, scenario_as_dict, scenario_to_markdown
│
├── scripts/
│   ├── run_evaluation.py         # metric batches over scenarios
│   ├── generate_report.py        # full results/ bundle (default seed 20260101)
│   └── benchmark.py              # runtime benchmark
│
├── tests/                        # 73 unittest tests
│   ├── test_quantum.py  test_teleportation.py  test_protocol.py
```

```
│   ├── test_detection.py  test_evaluation.py  test_attacks.py  test_cli.py
│
├── docs/
│   ├── technical_architecture.md # this document
│   ├── architecture.md  math.md  attacks.md  detection.md
│   ├── results.md  limitations.md  prd-traceability.md
│
└── results/                    # generated artifacts (seed 20260101)
    ├── config.json  evaluation.json  noise_sweep.json
    ├── group_sweep.json  scenarios.json  scenarios.md
```

---

# 18. References

### SIH Problem Statement

**Smart India Hackathon 2026, Problem Statement SIH26141 — Egreen Quanta LLP.** Defines a teleportation-based QDS threat-detection framework using Bell-state entanglement, teleportation, Pauli operations, projective measurements, statistical thresholding, attack simulation, security event logging and verification evaluation, **with AI/ML detection excluded**.

### Quantum Digital Signatures

**Dunjko, V., Wallden, P., & Andersson, E. (2014).** *Quantum Digital Signatures without Quantum Memory.* Physical Review Letters, 112, 040502. DOI: 10.1103/PhysRevLett.112.040502.

**Clarke, P. J., Collins, R. J., Dunjko, V., Andersson, E., Jeffers, J., & Buller, G. S. (2012).** *Experimental demonstration of quantum digital signatures using phase-encoded coherent states of light.* Nature Communications, 3, 1174. DOI: 10.1038/ncomms2172.

**Collins, R. J., Donaldson, R. J., Dunjko, V., Wallden, P., Clarke, P. J., Andersson, E., Jeffers, J., & Buller, G. S. (2014).** *Realization of Quantum Digital Signatures without the Requirement of Quantum Memory.* Physical Review Letters, 113, 040502. DOI: 10.1103/PhysRevLett.113.040502.

### Quantum Teleportation

**Nielsen, M. A. & Chuang, I. L., *Quantum Computation and Quantum Information* (Cambridge), Ch. 1** — canonical teleportation with two classical bits and conditional Pauli corrections; the reference implementation used to fix the correction convention `Z^m1 X^m2`.

### Implementation Notes

- The project is pure **Python + NumPy** (exact complex arrays and Born-sampled RNG). It does **not** use Qiskit, Qiskit Aer, a web framework, a database, or Docker.
- Measurement/threshold math is documented in `docs/math.md`; numeric results in `docs/results.md`.

---

## Final Architecture Classification

**Project type:** Software security research prototype **Core domain:** Quantum Digital Signatures **Quantum mechanism:** Bell-state entanglement + teleportation + Pauli corrections + projective measurements **Execution:** Exact classical simulation (Python + NumPy), ≤ 3-qubit registers **Threat detection:** Classical statistical + quantum-inspired Pauli-measurement threshold analysis **AI/ML: Not used Quantum advantage claim: None Real quantum hardware: None (simulator-only) Dataset:** Simulation-generated (no external dataset) **Interface:** Terminal CLI + JSON/Markdown reports (no web API/UI, no database, no Docker) **Deliverables provided:** Prototype, 73-test suite, generated `results/`, 8 documentation files, README, packaging (`pyproject.toml`, editable-install console script).

# 4. Mathematical Model

## Notation

- `L` - number of message bits (`message_length`, default 8).
- `n` - teleportation copies per message bit (`copies_per_bit`, default 4).
- `m`, `k` - message and shared key, both `L`-bit strings.
- `v_i = m_i ⊕ k_i` - the *signed value* at position `i`.
- `σ_i ∈ {X, Z}` - key-dictated measurement basis: `k_i = 0 → Z`, `k_i = 1 → X`.
- `|b⟩_σ` - eigenstate of Pauli `σ` with eigenvalue `(−1)^b`.
- `λ = (−1)^{sign_bit}` - Bell resource sign (default `+1`).

## Signature qubits

The sender prepares, for every position `i`, the qubit `|v_i⟩_{σ_i}`. The verifier knows `m` and `k`, hence `v_i`, hence both the basis and the target eigenvalue `(−1)^{v_i}` of every transmitted copy.

## Distribution by teleportation

Each of the `L·n` signature qubits is teleported through a *fresh* Bell pair

```
|Φ(λ)⟩_σ = ( |0,0⟩_σ + λ |1,1⟩_σ ) / √2
```

using the three-qubit circuit `CNOT(input → sender_half)`, `H(input)`, two computational-basis measurements, and the Pauli correction `Z^m1 X^m2` applied by the receiver. The two bits `(m1, m2)` travel over the *classical* control channel.

**Correlator identity.** For `|Φ(λ)⟩_σ`,

```
⟨σ ⊗ σ⟩ = +1
⟨τ ⊗ τ⟩ = λ      (τ = the other basis axis)
⟨Z ⊗ X⟩ = ⟨X ⊗ Z⟩ = 0
```

This is the pattern the *entanglement witness* checks against.

## Distillable amplification (parity checks)

Copies at one message position are grouped into consecutive groups of `g` (`parity_group_size`, default 2), `g = 2` giving the GHZ/cat-state picture. The products of each position's copies are *not* measured individually; instead the product observable

```
Π = σ ⊗ σ ⊗ … ⊗ σ   (g copies)
```

is measured. Because the group state factorises,

```
⟨Π⟩ = ∏_j ⟨σ_j⟩,
                P(parity = +1) = (1 + ⟨Π⟩)/2.
```

A single error in the group anticommutes with exactly one factor and therefore flips the parity **with probability 1**. This is the amplification: one parity check detects any *odd* number of errors inside its group regardless of `g`, while `s` independent per-copy checks miss a lone error with probability `(1 − q)^s`.

`g = 1` degenerates the parity check to a single-qubit check; the sweeps in the results report the measured trade-off between group size, mismatch rate and runtime.

## Error model

`pauli_decoherence` applies a uniformly random non-identity Pauli `P ∈ {X, Y, Z}` to a fraction of copies. `P` flips the eigenvalue of a `σ` eigenstate exactly when `P` anticommutes with `σ` — probability `2/3`. The *classical* tamper attack instead forges the correction bits, leaving a uniformly random residual Pauli that flips the eigenvalue with probability `1/2` — which is why one Pauli flip per copy has probability `1/2` of producing a mismatched signature eigenvalue.

`photon_loss` drops qubits outright; a per-qubit measurement cannot notice a missing qubit, so detection of loss is by *counting* copies (coverage check).

## Verification statistics

Every check compares an observed eigenvalue (or parity) to the target printed by the verifier's own record:

```
E = N_mismatch / N_all     mismatch rate over single + parity + witness + coverage
QBER = N_mismatch_single / N_single
```

The statistical detector trips if any of its scalar statistics exceeds the fixed threshold in `DetectionConfig` (defaults: `E > 0.05`, `QBER > 0.02`, per-basis `QBER > 0.05`, coverage shortfall `> 0`, freshness/authorisation violated).

## Pauli-measurement observables

For each basis the Pauli layer reports:

- **Eigenvalue agreement** - mean `⟨σ⟩` over copies targeted at the `+1` eigenstate (`+1` ideal).
- **Basis distribution distance** - total-variation distance between the observed and *expected* fraction of `+1` outcomes (`0` ideal). The expected fraction is derived from the signed message, not assumed uniform.
- **Parity agreement** - fraction of cat-state groups whose measured parity matched the group's intended parity (`1` ideal).

The **entanglement witness** is the fraction of retained Bell pairs whose four two-body correlators `ZZ, XX, ZX, XZ` still match the ideal pattern above.

## Detection limits (measured, not assumed)

Replay and unauthorised verification leave the measurement transcript bit-identical to a legitimate run. *No* measurement-based detector can see them; the statistical layer alone adds the classical freshness/authorisation rules. The evaluation reports this honestly as the Pauli layer's measured false negatives.

# 5. Threat Model and Attack Simulations

The problem statement's threat categories map onto five concrete attack classes plus the benign baseline. Every attack produces a declarative `AttackOutcome` and is executed by the shared `QuantumChannel` and `QDSSession` machinery.

| KEY | CLASS | WHAT IT DOES | MEASUREMENT-VISIBLE? |
|---|---|---|---|
| `legitimate` | `BenignExchange` | No adversarial action (false-alarm baseline) | n/a |
| `forgery` | `SignatureForgery` | Guesses the key bit-by-bit and forges qubits; wrong guesses are eigenstates of the wrong Pauli operator | yes |
| `impersonation` | `Impersonation` | Presents a well-formed signature bound to the attacker's *own* key while claiming to be the sender | yes |
| `replay` | `ReplayAttack` | Re-submits a previously accepted transaction (same message, key, nonce) | **no** |
| `unauthorized_verification` | `UnauthorizedVerification` | Genuine, unmodified transaction submitted by an entity outside the authorised set | **no** |
| `channel_manipulation` | `ChannelManipulation` | Injects Pauli errors, forges classical teleportation bits, drops qubits and disturbs the witness | yes |

## UC-1 Signature forgery

The adversary does **not** know the shared key, so it guesses. A guessed key matches the real one on roughly half the positions. Where the guess is wrong the forged qubit is an eigenstate of the wrong Pauli operator, and the verifier's own-basis measurement of it returns a random `±1` with probability 1. Detected by both the protocol (per-copy mismatch) and the Pauli layer (eigenvalue agreement collapses).

## UC-2 Sender impersonation

The impersonator has a *legitimate* key — its own — and uses it to build an internally consistent signature, relabelling the transaction as coming from the real sender. The signature is consistent with the wrong key, so mismatch concentrates exactly on the positions where the attacker's key differs from the verifier's. This is deliberately different from forgery: forging is random noise, impersonation is a *valid* signature bound to the wrong secret.

## UC-3 Replay attack

The adversary replays the quantum transcript of a previously accepted transaction. Every Pauli expectation and measurement outcome is reproduced exactly. **Replay is invisible to any measurement-based detector by construction**; it is caught by the classical freshness (nonce) cache in `ReplayCache`, which is part of the statistical layer's rule set. The evaluation reports the Pauli layer's failure to see this as an honest false negative.

## UC-4 Unauthorised verification

A genuine, unmodified transaction is submitted by a requesting entity outside the authorised set `{alice, victor}`. The quantum state is untouched and all measurements agree. Like replay, this is invisible to the Pauli layer and is caught by the authorisation rule in the statistical layer.

## UC-5 Quantum channel manipulation

The adversary attacks the transport rather than the protocol logic, using four primitive disturbances that model genuinely different attacker capabilities:

- **Pauli decoherence** (`pauli_decoherence`): random non-identity Pauli error on copies. Flips the target eigenvalue with probability 1 for an eigenstate.
- **Classical tampering** (`classical_tamper`): forges the two teleportation bits, so the receiver applies the wrong correction. The residual Pauli is random and flips the eigenvalue only half the time — genuinely weaker.
- **Photon loss** (`photon_loss`): drops copies; detected by *counting*.
- **Witness disturbance** (`witness_disturbance`): degrades the retained Bell pairs, attacking the entanglement witness itself.

The default `ChannelManipulation` uses 100% Pauli errors, 50% classical tampering, 0% loss, and 50% witness disturbance.

## Small-print honesty

- The channel simulation and the detection layers share the same vocabulary (`Disturbance` / statistical signals). Physical noise and adversarial activity are deliberately indistinguishable, so the false-alarm benchmark is honest.
- `expected_quantum_visible` is a *ground-truth* property recorded on each `AttackOutcome`; the evaluation compares the measured detection rate against it rather than assuming perfect detection.

# 6. Detection Design

Both layers consume exactly the same `DetectionInput` and are compared fairly. **Neither layer is AI or ML**: all signals are arithmetic, closed-form statistics of the measurement data or classical protocol rules, compared against fixed constants from `DetectionConfig`.

## Method 1 - statistical detection (`StatisticalDetector`)

Classical statistics of the verification run plus the two classical protocol rules whose failure no measurement can expose.

| SIGNAL | MEANING | TRIP RULE |
|---|---|---|
| `mismatch_rate` | `E = N_mismatch / N` over all check families | `> 0.05` |
| `qber` | per-copy quantum bit error rate over single-qubit family | `> 0.02` |
| `per_basis_qber_X` / `per_basis_qber_Z` | same rate per key-dictated basis | `> 0.05` |
| `coverage_shortfall` | fraction of expected copies never returned | `> 0` |
| `replay` | nonce already accepted | violated |
| `authorisation` | requesting entity not in authorised set | violated |

The last two are intentionally classical: replay and unauthorised verification leave the quantum data perfect, so only a protocol-context rule can see them.

## Method 2 - Pauli-measurement detection (`PauliMeasurementDetector`)

Restricted to **quantum measurement data and the verifier's record of the message**. It never reads nonces, identities or authorisation state.

| SIGNAL | MEANING | TRIP RULE |
|---|---|---|
| `eigenvalue_agreement_X/Z` | mean Pauli expectation over copies targeted at `+1` | `< 0.90` |
| `basis_distribution_distance_X/Z` | TV distance between observed and expected `+1` fractions | `> 0.20` |
| `parity_agreement_X/Z` | fraction of cat-state groups whose parity matched their intended parity | `< 0.90` |
| `entanglement_witness` | fraction of Bell pairs whose two-body correlators match the ideal | `< 0.90` |

Because replay and unauthorised verification produce measurement data identical to the legitimate baseline, the Pauli layer *correctly reports them as legitimate*. The results document that as the honest limit of measurement-based detection.

## Composite verdict

`DetectionEngine` runs both layers and reports:

- per-detector verdicts,
- agreement between them,
- the full per-basis measurement summary,
- the set of signals that tripped.

A transaction is considered a threat if **either** layer trips; the report always shows both verdicts so the reader can see how much the Pauli layer contributes on top of the classical rules.

## Why the two layers disagree (and why that is a feature)

The design intentionally surfaces where quantum measurements *can* and *cannot* establish an attack:

- forgery, impersonation, channel manipulation ⇨ both layers trip;
- replay, unauthorised verification ⇨ statistical-only (quantum-invisible).

The scenario table and the metrics in `docs/results.md` and `results/` report all of this quantitatively, including the Pauli layer's measured false negatives.

# 7. Results

Produced by `scripts/generate_report.py` with the default seed `20260101` and the default configuration (`L = 8`, `n = 4`, `witness_pairs = 8`, `parity_group_size = 2`). Raw artifacts: `results/config.json`, `results/scenarios.json`, `results/evaluation.json`, `results/noise_sweep.json`, `results/group_sweep.json`, `results/scenarios.md`.

## Method comparison (200 trials per scenario)

| DETECTOR | DETECTION RATE (95% CI) | FALSE-ALARM RATE | ACCURACY |
|---|---|---|---|
| statistical | **1.000** [0.996, 1.000] | **0.0000** | 1.000 |
| pauli measurement | **0.600** [0.569, 0.630] | **0.0000** | 0.667 |

Per-attack ground truth (statistical / Pauli detections out of 200):

| SCENARIO | TRUTH | STATISTICAL | PAULI | PROTOCOL REJECTS | MEAN E |
|---|---|---|---|---|---|
| legitimate | benign | 0 | 0 | 0 | 0.0000 |
| forgery | attack | 200 | 200 | 200 | 0.2344 |
| impersonation | attack | 200 | 200 | 200 | 0.2656 |
| replay | attack | 200 | 0 | 200 | 0.0000 |
| unauthorized verification | attack | 200 | 0 | 200 | 0.0000 |
| channel manipulation | attack | 200 | 200 | 200 | 0.5000 |

The Pauli layer's `0.600` is **not an implementation defect** - it is the measured consequence of the design: replay and unauthorised verification keep the quantum transcript bit-identical to a legitimate run, so no measurement-based detector can see them (see `docs/detection.md`). The statistical layer catches them via the classical freshness and authorisation rules.

## Noise sweep (legitimate false-alarm behaviour)

Decoherence rate → statistical detection / statistical false alarm / Pauli detection / Pauli false alarm (50 trials per point):

| P | STAT DETECTION | STAT FALSE ALARM | PAULI DETECTION | PAULI FALSE ALARM |
|---|---|---|---|---|
| 0.000 | 1.000 | 0.000 | 0.600 | 0.000 |
| 0.002 | 1.000 | 0.000 | 0.600 | 0.000 |
| 0.005 | 1.000 | 0.000 | 0.600 | 0.000 |
| 0.010 | 1.000 | 0.000 | 0.600 | 0.000 |
| 0.020 | 1.000 | 0.000 | 0.600 | 0.000 |
| 0.050 | 1.000 | **1.000** | 1.000 | **1.000** |

Interpretation: with a channel holding `L · n = 32` copies, a single flipped copy already trips the per-copy QBER rule. At `p = 0.05` the expected number of flipped copies is `32 · 0.05 = 1.6`, so essentially every *legitimate* transaction is rejected - the operating point collapses entirely. The design target stated in `DetectionConfig` ("false alarms near 1% while `p ≤ 0.01`") is met: below `p = 0.02` the false-alarm rate is 0 in this fixed-seed run.

## Group-size sweep (distillable amplification)

`g` → Pauli detection / statistical false alarm / mean mismatch rate over legitimate + channel-attack trials / runtime (50 trials per point):

| G | STATISTICAL FALSE ALARM | PAULI DETECTION | MEAN E | MEAN RUNTIME |
|---|---|---|---|---|
| 1 | 0.000 | 1.000 | 0.2750 | 12.2 ms |
| 2 | 0.000 | 1.000 | 0.2500 | 12.0 ms |
| 3 | 0.000 | 1.000 | 0.2500 | 11.8 ms |
| 4 | 0.000 | 1.000 | 0.2321 | 11.7 ms |

All group sizes detect the channel attack perfectly with zero false alarms. Increasing `g` measurably lowers the mean mismatch rate (the parity check collapses whole groups rather than individual copies) and slightly reduces runtime (fewer measurements). `g = 1` degenerates to independent per-copy checks; the fixed protocol default `g = 2` sits in the stable middle of the sweep.

## Runtime benchmark

Mean wall-clock per scenario over 20 repetitions (`scripts/benchmark.py`):

```
legitimate                    11.25 ms
forgery                       11.05 ms
impersonation                 10.73 ms
replay                        11.03 ms
unauthorized_verification     11.12 ms
channel_manipulation          11.91 ms
OVERALL                       11.18 ms  (120 transactions)
```

## Reproducibility

Everything is seeded from `ExperimentConfig.seed` (`20260101`); every RNG stream is derived from the seed plus a per-transaction offset. Rerun with:

```
python scripts/generate_report.py --trials 200 --sweep-trials 50
```

# 8. Interface & Frontend Specification

**Smart India Hackathon 2026 — SIH26141** · **Egreen Quanta LLP** · **Category: Software** · **Theme: Security**
**Project:** *Quantum-Inspired Cyber Threat Detection for Digital Signature Security* **Document Type:** Frontend / Interface Specification · Status: Prototype Documentation · Version: 1.0

> **Scope statement (important).** QuantumSigGuard's **primary** interface is the **terminal** provided by `qsentinel/cli.py` plus the **reporting/export layer** (`qsentinel/reporting/`) that renders results as ANSI console output, JSON and Markdown. An **optional local web viewer** (`streamlit run streamlit_app.py`, under the `[web]` extra) mirrors those results as a read-only dashboard — it is a pure view layer over the same engine and adds no detection logic (§18.7, §21). There is no REST API, no deployed URL, no SPA framework and no browser-side state. This document therefore specifies the interface that **actually exists**, and marks every browser/design-system/API item from a conventional "Frontend Specification" as **not implemented / future roadmap** rather than inventing a stack.
> The SIH evaluator experience is a **three-minute terminal demo** (§2.3, §4.3) — the export layer provides the evidence artifacts (`results/*.json`, `results/*.md`), and the optional dashboard is a convenience viewer, not a required path.

# 1. Overview & Scope

## 1.1 Purpose

The interface makes the full workflow operable and readable without a browser:

```
Create QDS Session (QDSSession)
      ↓
Generate / Simulate Signature (sender sign)
      ↓
Optional Attack Injection (attacks registry)
      ↓
Verify Signature (verifier.verify)
      ↓
Analyze Quantum Measurements (verification checks + measurement_summary)
      ↓
Apply Statistical / Threshold Detection (two detectors)
      ↓
Classify Verification Result (DetectionEngine)
      ↓
Inspect Security Metrics and Logs (render_*  +  JSON/Markdown export)
```

Every screen-equivalent below is a rendered console view or exported artifact whose content comes **only** from computed results — the interface never invents or alters a detection decision (§18.7).

## 1.2 Interface Capability Register (actual status)

| CAPABILITY | WHERE IT LIVES | STATUS |
|---|---|---|
| QDS overview "dashboard" | `evaluate` + `scenarios` console output | **Implemented** (console) |
| Teleportation-based QDS simulation | `qsentinel-cli demo`, `run_demo.py` | **Implemented** |
| Signature generation | protocol sender, shown in `render_scenario` | **Implemented** |
| Signature verification | `render_scenario` "Verification" block | **Implemented** |
| Attack simulation | `demo <attack>` / `scenarios` / `evaluate` | **Implemented** |
| Statistical threat detection | statistical signal table | **Implemented** |
| Quantum-inspired (Pauli) detection | Pauli signal table + measurement summary | **Implemented** |
| Experiment comparison | `render_comparison` method table | **Implemented** |
| Security-event logging | `results/scenarios.json`, event export | **Implemented** |
| Export / report view | `--output` JSON, `scenarios.md`, `generate_report.py` | **Implemented** |
| Web dashboard (view) | `streamlit run streamlit_app.py` (optional `[web]` extra) | **Implemented** (read-only, local) |
| Web dashboard / SPA | — | **Not implemented — future roadmap (§21)** |
| REST API | — | **Not implemented** |
| Real quantum hardware view | — | Out of scope (§21) |
| AI/ML detection | — | **Explicitly excluded** |

## 1.3 Assumptions (verified, not TBD)

| ITEM | VERIFIED STATUS |
|---|---|
| Python-based QDS prototype | **Implemented** (`qsentinel/`) |
| Teleportation-based QDS simulation | **Implemented** (`protocol/teleportation.py`) |
| Attack simulation (5 classes + benign) | **Implemented** (`attacks/`) |
| Statistical + Pauli-measurement detection | **Implemented** (`detection/`) |
| AI/ML | **Not used** |
| Real quantum hardware | **Not claimed** |
| Web frontend framework | **Streamlit viewer** (optional `[web]` extra, read-only, local) |
| REST API paths | **None exist** (CLI contract instead, §10) |
| Authentication | **None needed** (local single-user CLI; dashboard is localhost-only) |
| Hosting URL | **None** (local process) |

### Stack Register (actual)

| AREA | ACTUAL IMPLEMENTATION |
|---|---|
| Interface shell | `qsentinel/cli.py` (`argparse`), console scripts `qsentinel-demo`, `python3 -m qsentinel.cli` |
| Language | Python 3.10+ (verified on 3.14.7) |
| "Styling" | ANSI color helpers — `qsentinel/reporting/console.py` (`NO_COLOR` / `FORCE_COLOR` aware) |
| Visualization | alignment-safe console tables + filled bars + Markdown tables (`reporting/`) |
| State management | `QDSSession` (replay cache, authorisation set, transaction counter) — not a browser store |
| API communication | none — direct in-process calls |
| Export | JSON (`export.write_json`) + Markdown (`scenario_to_markdown`, `generate_report.py`) |
| Hosting | local terminal only |

**No new library was (or should be) introduced to satisfy this document.**

---

# 2. Users, Personas & Key Use Cases (CLI-anchored)

## 2.1 Primary Personas

### Security Analyst

- Run verification: `qsentinel-demo demo legitimate` vs `demo replay` — read the `E` mismatch rate and the tripped signals.
- Inspect *why* a run flagged: each detector prints a **signal/value/rule/threshold/state** table (`render_scenario`).
- Compare runs: `qsentinel-demo scenarios` — side-by-side **truth / statistical / pauli / protocol / E** table (`render_comparison`).

### Researcher / Technical Evaluator

- Configure: `--seed`, `--preset` (`clean`, `noisy`, `degraded`, `unamplified`), `--trials`.
- Measure detector behaviour: `evaluate`, `noise-sweep`, `group-sweep` with `--output` JSON.
- Read exact measurement statistics: per-basis `measurement_summary` (copies, expected/observed +1, distribution distance, parity agreement) in `render_scenario`.

### SIH Judge / Demo Viewer

- Three-minute path (see §2.3): one legit run → one attacked run → comparison + event evidence.
- Read values, not jargon: verdict labels are **ACCEPT / REJECT / THREAT / LEGITIMATE** in text, never bare numbers or colors alone (§14.2).

### Administrator

**Not implemented** — single-operator local tool; no accounts. Becomes relevant only if a web service is added (§21).

### 2.2 Persona User Stories (mapped)

- *As a security analyst I want to inspect measurement deviations so I understand why a run was flagged.* → `demo <attack>` prints value-vs-threshold per signal and the per-basis summary.
- *As a researcher I want a reproducible experiment.* → a single `--seed` parameterizes every run; `results/config.json` records it.
- *As a judge I want legit-vs-attacked side-by-side.* → `scenarios` comparison table.

---

## 3. Information Architecture

### 3.1 Command Tree (the "sitemap")

```
qsentinel (prog; --version prints 1.0.0)
│
├── scenarios  [--seed N] [--preset NAME] [--json]
│      run all six scenarios once; comparison table (or JSON)
│
├── demo  [scenario]  [--seed N] [--preset NAME]
│      full per-transaction report; default steps through all six
│
├── evaluate  [--trials N] [--seed N] [--preset NAME] [--output PATH]
│      confusion-matrix metrics over N trials per attack
│
├── noise-sweep  [--rates "a b c ..."] [--trials N] [--seed] [--preset] [--output]
│      detection vs channel decoherence
│
└── group-sweep  [--sizes "g1 g2 ..."] [--trials N] [--seed] [--preset] [--output]
       detection vs distillation group size
```

### 3.2 Export Artifacts ("screens" stored on disk)

```
results/
├── config.json        # full configuration incl. seed (reproducibility record)
├── scenarios.json     # per-scenario security-event records (see §5.2)
├── scenarios.md       # self-contained Markdown report of all six scenarios
├── evaluation.json    # metrics + per-attack table + runtime
├── noise_sweep.json   # detection/FAR vs decoherence
└── group_sweep.json   # detection/FAR vs parity group size
```

### 3.3 Route Table

**Not applicable** — there are no HTTP routes. The equivalent is the command table in §3.1 and the export artifacts in §3.2.

---

## 4. User Flows

### 4.1 Standard Verification Flow

```
flowchart TD
    A[qsentinel demo legitimate] --> B[Configure: seed / preset]
    B --> C[QDSSession new transaction]
    C --> D[sign + distribute n copies]
    D --> E[teleport + Pauli correction]
    E --> F[projective measurement + verification]
    F --> G[run both detectors]
    G --> H{result}
    H -->|accepted| I[ACCEPT + LEGITIMATE labels]
    H -->|rejected| J[REJECT + THREAT + reject_reason]
    I --> K[event exported to JSON]
    J --> K
```

### 4.2 Attack Simulation Flow

```
flowchart TD
    A[qsentinel demo forgery | replay | ...] --> B[attack selected from registry]
    B --> C[AttackContext built: txn, rng, cache, authorisation]
    C --> D[attack.run -> AttackOutcome]
    D --> E[channel.transmit with outcome.disturbances]
    E --> F[verify + detect]
    F --> G[detection result table]
    G --> H[compare against legitimate baseline]
    H --> I[security event stored]
```

### 4.3 Judge Demo Flow (three minutes)

```
flowchart LR
    A[python run_demo.py] --> B[scenarios: legit row ACCEPT]
    B --> C[scenarios: replay row REJECT + THREAT]
    C --> D[demo replay: show E + tripped signal replay]
    D --> E[open results/scenarios.md + events]
```

---

## 5. Screen-by-Screen Specification (console views)

### 5.1 "Dashboard" — `scenarios` comparison table + `evaluate` summary

**Purpose:** high-level overview without low-level detail.

Real output (`render_comparison`, seed = run at report generation):

```
Method comparison
-------------------------
scenario        truth  statistical  pauli       protocol  E
-------------  ------  -----------  ----------  --------  ------
legitimate     legit   LEGITIMATE   LEGITIMATE  ACCEPT    0.0000
forgery        attack  THREAT       THREAT      REJECT    0.1562
impersonation  attack  THREAT       THREAT      REJECT    ...
replay         attack  THREAT       LEGITIMATE  REJECT    0.0000
```

`evaluate` adds the aggregate "metric cards": per-detector **detection rate (95% CI), false-alarm rate (95% CI), accuracy**, a filled status bar, per-attack counts and mean runtime.

## 5.2 QDS Simulation "screen" — `qsentinel-demo demo [scenario]`

Full per-transaction report (`render_scenario`). Genuine excerpt (forgery run, seed default):

```
Scenario: Signature forgery  [forgery]
  Adversarial action: Guessed 4/8 key bits correctly. ...
  Ground truth: ATTACK   quantum-visible: True

  Protocol data
    transaction 55fcfbb22e008524  nonce 212124108139  message 00110001  length 8
    requester 'alice'  authorised: True
    basis pattern ZXZXZXXZ  (key redacted; target eigenvalues +--++---)

  Distribution
    32 teleportation copies  8 witness pairs  0 dropped  0 classical messages tampered
    residual Pauli errors: I=32

  Verification
    signature valid: THREAT    freshness: LEGITIMATE    authorisation: LEGITIMATE
  check family  checks  failed  pass rate
  ------------  ------  ------  ---------
  single            32       7      78.1%
  parity            16       7      56.2%
  witness            8       0     100.0%
```

Per-basis measurement chart-equivalent:

```
basis  copies  expected +1  observed +1  distance  parity
-----  ------  ------------  ------------  --------  ------
X         16        50.0%         37.5%    0.1250   0.750
```

No fabricated fidelity or security percentages are shown — only computed values (runtime, mismatch rate, distances, parities).

## 5.3 Attack Simulation "screen" — `demo <attack>` / `scenarios`

For each attack, the report shows: adversarial action notes, ground truth, `expected_quantum_visible`, distribution disturbances, reject reason and detection evidence. The five attack keys: `forgery`, `impersonation`, `replay`, `unauthorized_verification`, `channel_manipulation`. `channel_manipulation` additionally enumerates its four disturbance channels (§ `docs/attacks.md`).

## 5.4 Verification Analysis — same `render_scenario` "Verification" block

Shows `signature valid / freshness / authorisation`, the check-family table, overall `E = mismatch/total`, and `reject_reason`. The decision is text: `THREAT` / `LEGITIMATE` (statistical and Pauli layers) plus `ACCEPT` / `REJECT` (protocol outcome) — readable in under one second.

## 5.5 Threat Analysis — detector evidence tables

Two tables, **Method 1: statistical** and **Method 2: Pauli measurement**, each row:

```
signal          value   rule  threshold  state
mismatch_rate   0.2000  >     0.0500     TRIPPED
replay          1.0000  >     0.0000     TRIPPED
...

basis_distribution_distance_Z   0.5000  >  0.2000  TRIPPED
parity_agreement_X              0.4375  <  0.9000  TRIPPED
entanglement_witness            1.0000  <  0.9000  ok
```

### 5.6 Experiment History — `evaluate --output evaluation.json`

Machine-readable per-attack table (attack, trials, statistical_detected, pauli_detected, protocol_rejected, mean E) plus aggregate matrices — the durable record of past runs.

### 5.7 Security Event Log — `results/scenarios.json`

Each event (per scenario) records: `scenario`, `title`, `is_attack`, `notes`, `expected_quantum_visible`, `accepted_by_protocol`, `elapsed_seconds`, `comparison` (statistical/pauli/agree), the full `verification` block (signature_valid, freshness_ok, authorisation_ok, reject_reason, mismatch_count, check_count, mismatch_rate, failing_checks), and `detection_measures` (every signal name/value/threshold/comparison/tripped).

### 5.8 About / System — `README.md` and command `--version`

Contains SIH26141, organization, project title, scope, no-AI/ML statement, simulated-only status, attack list and limitations.

---

## 6. Component Library (console "components")

| COMPONENT | REAL IMPLEMENTATION | RENDERS |
|---|---|---|
| `heading(text)` | `reporting/console.py:55` | bold underlined section title |
| `rule()` | `console.py:60` | 78-char rule |
| `table(headers, rows, aligns)` | `console.py:65` | alignment-safe fixed-width table; returns `(no rows)` when empty |
| `verdict_label(b)` | `console.py:96` | `THREAT` / `LEGITIMATE` (text label; color optional) |
| `bar(fraction, width)` | `console.py:103` | `####....` filled bar (status visualization) |
| `colour(text, style)` | `console.py:42` | ANSI style; disabled unless tty / `NO_COLOR` |
| `terminal_width()` | `console.py:110` | responsive-ish column sizing |
| `render_scenario / render_comparison / render_evaluation` | `reporting/render.py` | page-level views |
| `scenario_as_dict / scenario_to_markdown / write_json` | `reporting/export.py` | JSON/Markdown exports |

There is **no DOM/component framework**; these functions are the "component library", kept free of business logic (`console.py` module docstring).

---

# 7. Design System

## 7.1 Semantics (ANSI colors actually used)

| SEMANTIC | STYLE TOKEN | USED FOR |
|---|---|---|
| threat | red | tripped signals, THREAT, REJECT |
| legitimate | green | ok signals, LEGITIMATE, ACCEPT |
| warn | yellow | reject_reason, DISAGREE |
| info | blue | protocol/distribution headings |
| accent | magenta | detector headings |
| muted | dim | rules, summary labels |

Color is **decorative**: every meaning is carried by text (`THREAT`, `TRIPPED`, `ok`, `REJECT`, `ACCEPT`) as well (§14.2). `NO_COLOR` strips ANSI codes entirely.

## 7.2 Typography & Layout

Terminal monospace; headings bold; numeric columns right-aligned; "responsive" via `terminal_width()` (best-effort). The optional Streamlit dashboard (§21.3) carries full CSS/design tokens in `ui/theme.py` / `.streamlit/config.toml` (dark-first palette, Inter + JetBrains Mono, WCAG AA contrast on the token pairs) — the console colors above map 1:1 onto those tokens.

## 7.3 Motion

**None** — console output is static text. Accessibility requirement (`prefers-reduced-motion`) is trivially satisfied (§14).

---

# 8. Data Visualization Specification

Everything shown is a **table, count, or filled bar** — the correct visualization for discrete measurement outcomes, and the export layer reuses the same primitives.

| CONCEPT | REALIZATION | REAL FUNCTION |
|---|---|---|
| Measurement distribution | per-basis table: expected vs observed +1 fraction, distance, parity | `render_scenario` (uses `measurement_summary`) |
| Detection threshold plot | signal table: `value rule threshold state` | `render_scenario` detector blocks |
| Legit vs attacked | comparison table: `truth statistical pauli protocol E` | `render_comparison` |
| Metrics | detection/FAR/accuracy with 95% CIs + filled `bar()` | `render_evaluation` |
| Timeline/event log | structured JSON event records | `scenario_as_dict` |
| Protocol flow | documented stages (`docs/math.md`) — never implies real hardware | doc text |

JSON data shapes (canonical, machine-readable) — e.g. detector measure:

```
{
  "name": "mismatch_rate",
  "value": 0.0,
  "threshold": 0.05,
  "comparison": ">",
  "tripped": false
}
```

evaluation summary shape includes per-detector `detection_rate`, `false_alarm_rate`, `accuracy` and `*_ci95` arrays plus `per_attack` rows and `mean_runtime_seconds`.

---

# 9. State Management & Data Flow

## 9.1 Persistent ("shared") state — `QDSSession`

| STATE | OWNER | PURPOSE |
|---|---|---|
| `replay_cache` | session | freshness across transactions (§ `protocol/freshness.py`) |
| `authorised_entities` | session | verification authorization set |
| `_accepted` history | session | record of accepted transactions (used by replay/eval seeding) |
| `_counter` | session | per-transaction RNG seed offset |
| `config` | session | immutable `ExperimentConfig` |

## 9.2 Flow (no asynchronous machinery)

```
sequenceDiagram
    actor User
    participant CLI as qsentinel.cli
    participant Session as QDSSession
    participant Engine as DetectionEngine
    participant Export as reporting/export

    User->>CLI: command + flags
    CLI->>Session: run_scenario(key)
    Session->>Session: attack.run(ctx) + channel.transmit
    Session->>Engine: build_input(verify report, config, authorisation)
    Engine-->>Session: DetectionReport (both verdicts)
    Session-->>CLI: ScenarioResult
    CLI->>Export: render_* / scenario_as_dict / write_json
    Export-->>User: tables, JSON, Markdown
```

No Redux/Zustand/Context equivalents exist or are needed — the session is the store.

---

# 10. Interface Contract (CLI, not REST)

## 10.1 Command Contract

| COMMAND | PURPOSE | FLAGS | OUTPUT | EXIT/ERROR |
|---|---|---|---|---|
| `scenarios` | all six scenarios once | `--seed`, `--preset`, `--json` | comparison table or JSON array | 0; `SystemExit("error: ...")` on bad input |
| `demo [scenario]` | full per-run report | `--seed`, `--preset` | rendered scenario report(s) | 0; `KeyError` on unknown scenario |
| `evaluate` | metrics over N trials/attack | `--trials` (1..10000), `--seed`, `--preset`, `--output PATH` | printed summary or JSON file | 0 |
| `noise-sweep` | detection vs decoherence | `--rates`, `--trials`, `--seed`, `--preset`, `--output` | sweep table or JSON | 0 |
| `group-sweep` | detection vs group size | `--sizes` (positive ints), `--trials`, `--seed`, `--preset`, `--output` | sweep table or JSON | 0 |
| `--version` | version | — | `1.0.0` | 0 |

## 10.2 Canonical JSON Contracts (what a future web layer would consume)

These are the *actual* export shapes (`ScenarioResult.as_dict`, `EvaluationResult.summary`, `scenario_as_dict`), defined in the codebase:

```
// scenario event (results/scenarios.json)
{
  "scenario": "replay",
  "is_attack": true,
  "accepted_by_protocol": false,
  "comparison": { "statistical": true, "pauli_measurement": false, "agree": false },
  "verification": {
    "signature_valid": true, "freshness_ok": false, "authorisation_ok": true,
    "reject_reason": "nonce replay", "mismatch_count": 0, "check_count": 64,
    "mismatch_rate": 0.0, "failing_checks": []
  },
  "detection_measures": { "replay": {
    "name": "replay", "value": 1.0, "threshold": 0.0, "comparison": ">", "tripped": true } }
}
```

```
// evaluation summary (evaluate --output)
{
  "statistical": { "true_positives": 1000, "false_positives": 0,
                   "detection_rate": 1.0, "detection_rate_ci95": [0.9962, 1.0],
                   "false_alarm_rate": 0.0, "accuracy": 1.0, ... },
  "pauli_measurement": { "detection_rate": 0.6, "detection_rate_ci95": [0.5693, 0.6299], ... },
  "per_attack": [ { "attack": "legitimate", "is_attack": false, "trials": 200, ... }, ... ],
  "mean_runtime_seconds": 0.0111, "total_runtime_seconds": 13.32
}
```

TypeScript interfaces, pagination, timeouts, retries etc. are **future web-only concerns** and are not invented here.

---

## 11. Handling Long-Running Computations

Simulations are **synchronous and fast** (measured, seed `20260101`, single-threaded):

| WORKLOAD | MEASURED WALL TIME (THIS HOST) |
|---|---|
| One transaction (any scenario) | ≈ 11 ms mean |
| `evaluate --trials 200` (1200 transactions) | ≈ 13.3 s |
| `generate_report.py` (full `results/` bundle, 200-trial evaluation) | ≈ 52 s |
| Test suite (`unittest discover`) | ≈ 4.4 s |

Best practice followed: no fake progress. Commands block and print final results. There is no cancellation, no polling and no partial-result mode — none is needed at this scale. If a web layer is added with large batch jobs, a job-ID/polling pattern would come first (roadmap §21).

---

## 12. Forms, Validation & Input Handling

All "forms" are CLI flags; every value is validated *twice* — at the CLI boundary and again in the frozen config dataclasses (backend is authoritative):

| FIELD | RULE | VALIDATOR |
|---|---|---|
| `--seed` | positive int | `cli` / `config` |
| `--trials` | integer `1..10000` | `cli._parse_trials_safe` |
| `--rates` | comma/space floats | `cli._parse_float_list` |
| `--sizes` | positive integers | `cli.command_group_sweep` |
| `--preset` | one of `clean,noisy,degraded,unamplified` | argparse choices |
| message/config values | `[0,1]` ranges, positive counts, `group <= copies`, `sign_bit ∈ {0,1}` | `config.py __post_init__` ×3 |

Error messages are specific (e.g. `error: expected a positive integer, got 'x'`); invalid scenarios raise `KeyError`; there is no dataset upload (§12.1).

### 12.1 Dataset Upload

**N/A** — no external dataset and no upload feature (SIH lists Public/Open data, but this implementation generates its observations from the seeded protocol). A file uploader is **not** added just to satisfy a "dataset" label.

---

## 13. Responsive Layout & Cross-Browser

**N/A** — there is no browser. The terminal renders alignment-safe fixed-width tables (`console.table`), honors `terminal_width()`, and strips ANSI when piped. Desktop/laptop terminal is the sole target; this matches the SIH demo format.

---

## 14. Accessibility & Internationalization

### 14.1 WCAG-relevant posture (console)

- **Status is never color-only**: `verdict_label()` and tables emit `THREAT` / `LEGITIMATE`, `TRIPPED` / `ok`, `ACCEPT` / `REJECT` as text; color is additive.
- Output degrades to plain text under `NO_COLOR` and when piped (`sys.stdout.isatty()`).
- No keyboard trap, no timed content, no flashing motion.
- Sample screen-reader-equivalent descriptions are trivially derived because all output is text.

### 14.2 Internationalization

**English only (implemented).** A future dashboard should add `en` / `hi` locales while keeping technical identifiers untranslated (Pauli-X/Y/Z, Bell state, QDS). Not implemented today.

---

## 15. Performance Budget & Optimization

### 15.1 Measured targets (this host)

| METRIC | VALUE |
|---|---|
| Verification latency per transaction | ≈ 11.1 ms |
| Throughput (single-threaded) | ≈ 90 transactions/s |
| Full evaluation (1200 trials) | ≈ 13.3 s |
| Full report bundle | ≈ 52 s |
| Test suite | ≈ 4.4 s (73 tests) |

| BROWSER METRIC | STATUS |
|---|---|
| Initial page load / Lighthouse | **N/A — no web app** |

### 15.2 Real optimization strategy

- Registers ≤ 3 qubits (constant per-run simulation cost).
- Vectorized NumPy statistics.
- Independent experiments → batch/`multiprocessing` parallelism is the documented next step (currently single-threaded).

---

## 16. Security on the Interface Side

- **No browser, no client secrets**: there are no tokens, credentials or storage to protect (§`docs/security.md` — verified by repo scan).
- **The interface is not the decision-maker**: verdicts come from `DetectionEngine`; `render_*` and `export` only serialize existing `DetectionReport` signals. The integrity rule of §18.7 is therefore structural, not aspirational.
- Errors shown are the CLI's own messages; no Python tracebacks are rendered for normal users (argparse `SystemExit` prints a short message; `KeyboardInterrupt` exits with code 130).
- Simulation vs real-hardware terminology is strictly separated in every view (§8, §5.8).

---

## 17. Error Handling

| ERROR CLASS | BEHAVIOUR |
|---|---|
| Bad flag / value | argparse/cli prints `error: <specific message>`, exit ≠ 0 |
| Invalid preset/scenario | argparse choices / `KeyError` (tested: `test_unknown_scenario_is_an_error`) |
| Config invariant violation | `ValueError` from `__post_init__` |
| Interrupt | `KeyboardInterrupt` → exit 130 |
| Uncaught exception | Python exception (local tool); never exposed through any network surface |

Logging: no third-party client analytics platform exists; application security events are computed and exported by the backend (`results/scenarios.json`), never fabricated locally (§`docs/security.md` §10.1).

---

# 18. Testing Strategy (implemented for the interface)

| TEST FILE | COVERS (INTERFACE-RELEVANT) |
|---|---|
| `tests/test_cli.py` | `scenarios`, `demo <attack>`, `evaluate`, `noise-sweep` commands; `--version`; JSON round-trip; `write_json` pretty-print; `render_scenario` non-empty with THREAT; unknown-scenario error |

### Mapping of conventional FE cases

| ID | SCENARIO | STATUS IN CLI WORLD |
|---|---|---|
| FE-03 | Execute legitimate run | `demo legitimate` → ACCEPT shown (tested) |
| FE-04 | Execute an implemented attack | `demo forgery` → THREAT shown (tested) |
| FE-02/06 | Invalid input / backend error | `cli` validation + `KeyError` (tested) |
| FE-01 | Empty state | model: no prior runs → `scenarios` still renders all rows; N/A for dashboards |
| FE-10 | Threshold/value graph matches backend | signal tables read directly from `DetectionReport` signals — no client-side recalculation |

### 18.7 Integrity test (non-negotiable)

If the backend verdict is `"threatDetected": false`, the rendering **must not** display a threat. Because `scenario_as_dict`/`render_scenario` consume `DetectionReport` booleans directly, the interface cannot invert a verdict.

---

# 19. Build, Deployment & Environment Setup

## 19.1 Environment Variables

**None required.** No `.env`, no credentials, no client-side configuration. Runtime dependency: NumPy only.

## 19.2 Run Instructions (real)

```
cd ~/Desktop/QuantumSentinel
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt            # numpy>=1.24
pip install -e .                            # optional: installs qsentinel-demo

python3 -m qsentinel.cli scenarios          # or: qsentinel-demo scenarios
python3 -m qsentinel.cli demo forgery
python3 run_demo.py                         # printed walkthrough
python3 scripts/generate_report.py          # regenerates results/
python3 -m unittest discover -s tests       # 73 tests
```

### 19.3 Deployment Status (real)

| ITEM | STATUS |
|---|---|
| Local development | Implemented (CLI) |
| Public URL / cloud / Docker / CI | **Not present** (local tool; future roadmap) |
| HTTPS | N/A (no network surface) |

## 20. Project Structure (interface-relevant subset)

```
QuantumSentinel/
│
├── qsentinel/
│   ├── cli.py                  # command surface (argparse)
│   ├── session.py              # orchestration + stateful store
│   ├── reporting/
│   │   ├── console.py          # tables, headings, colors, bars, verdict labels
│   │   ├── render.py           # render_scenario / render_comparison / render_evaluation
│   │   └── export.py           # write_json, scenario_as_dict, scenario_to_markdown
│   ├── detection/  evaluate/  attacks/  protocol/  quantum/   # compute layers
│   │
├── scripts/
│   ├── generate_report.py      # builds results/ bundle
│   ├── run_evaluation.py
│   └── benchmark.py
│
├── tests/test_cli.py           # interface tests
├── run_demo.py                 # narrative walkthrough
├── results/                    # scenarios.json/md, evaluation.json, sweeps, config.json
└── docs/
    ├── frontend.md             # this document
    ├── technical_architecture.md  security.md  architecture.md  math.md
    ├── attacks.md  detection.md  results.md  limitations.md  prd-traceability.md
```

Naming conventions: Python (snake_case modules/functions, PascalCase classes) — not JavaScript. Quantum math stays out of presentation code (separation enforced by `reporting/` importing only dataclasses).

# 21. Limitations & Future Frontend

## 21.1 Current Interface Limitations

- Primary interface is console + exports; the optional Streamlit dashboard (§21.3) is read-only and local, with no REST API or persistence of its own.
- No dedicated HLA/"verification UI" beyond console rendering; judge demo is terminal-based.
- Dashboard charts are coarse (bar/line only), no interactive threshold manipulation, no browser-side state.
- No multilingual (English-only), no graphical threshold plots beyond the dashboard's value bars.
- No large-batch long-running jobs (not needed at 90 tx/s).

## 21.2 Roadmap

**Phase 1 — SIH prototype (done):** CLI flows above, event export, judge demo path, optional local web dashboard (§21.3).

**Phase 2 — web dashboard in a browser (partially built as the local viewer):** a lightweight read-only UI over the existing JSON contracts (`evaluation summary`, `scenario event`, `detection_measures`) — already started via `streamlit_app.py` + `app_pages/` + `ui/`. Remaining roadmap items: interactive threshold plots (value-vs-threshold sliders), per-basis measurement distribution bars, legit-vs-attacked comparison, WCAG AA tokens and `en`/`hi` locales if adopted; a REST layer only if a service deployment is required. No fabrication of progress (real job blocks/polls), no invention of security metrics.

**Phase 3 — scale (only if warranted):** bigger workloads (multiprocessing), richer attack models, real-parameter batch jobs.

Throughout: the strict "no invented values, no real-quantum implication, no AI/ML, backend is source of truth" rules of this document carry over unchanged.

## 21.3 Local Web Dashboard (implemented)

An **optional** Streamlit dashboard ships under the `[web]` extra. It is explicitly a **view layer**:

- `streamlit_app.py` — entry point, branded sidebar, global styling injection (`ui/theme.py`), footer; no state beyond Streamlit's own.
- `app_pages/overview.py` — KPIs, method comparison, per-attack counts, detection-rate bar chart and both robustness sweeps, all read from `results/*.json` (values are never recomputed or hard-coded; when the bundle is absent the page shows an empty state).
- `app_pages/scenario_lab.py` — runs one scenario live through the real engine (`QDSSession`, seeded), shows the classical protocol verdict, per-signal value-vs-threshold progress bars for both detectors, and the full failed-check list.
- `app_pages/evaluation.py` — live `run_evaluation` with chosen trials/seed and JSON/CSV export.
- `web_helpers.py` — pure reshaping/format helpers (DataFrames, CSV/JSON payloads); engine logic stays in `qsentinel/`.
- `ui/components.py` + `ui/theme.py` — design tokens (dark-first palette, Inter + JetBrains Mono), a matching Altair theme, and reusable components (hero, KPI cards, verdict badges, styled tables, empty states, callouts). `.streamlit/config.toml` defines both dark and light themes.

Design tokens (dark): bg `#0B1220`, surface `#111A2E`, accent cyan `#22D3EE`, violet `#8B5CF6`, safe `#22C55E`, threat `#EF4444`, warning `#F59E0B`, info `#38BDF8`. Footers carry `SIH 2026 · SIH26141 · Egreen Quanta LLP`.

---

# Final Interface Position

The QuantumSigGuard interface tells one story — printable on a terminal in three minutes:

```
flowchart LR

A[Run QDS flow] --> B[Teleportation + Pauli correction]
B --> C[Verification]
C --> D[Measurements]
D --> E[Statistical + Pauli detection]
E --> F{Threat?}
F -->|No| G[ACCEPT / LEGITIMATE]
F -->|Yes| H[REJECT / THREAT + evidence]
G --> I[event in scenarios.json]
H --> I
```

The interface prioritizes **explainability over decoration**: for every transaction it shows not only the verdict but the exact signal values, rules, thresholds and tripped states behind it, and it persists that evidence as JSON/Markdown for judges and evaluators. That aligns with the SIH requirement for a teleportation-based QDS framework incorporating signature verification, controlled cyber-attack simulation, Pauli/projective measurement analysis, statistical threat identification, security analysis, performance evaluation and security-event logging — **without AI/ML and without claiming a web stack that does not exist.**

# 9. Security & Access Document

**Smart India Hackathon 2026 — Problem Statement SIH26141 · Egreen Quanta LLP · Category: Software · Theme: Security Project:** *Quantum-Inspired Cyber Threat Detection for Digital Signature Security* **Document Type:** Security & Access · Status: Prototype Documentation / SIH Submission · Version: 1.0

> This document describes the security and access posture of the **implemented** prototype **QuantumSigGuard** (`~/Desktop/QuantumSentinel`). Every status below is corroborated by source code and measured output referenced in the **Evidence map (§13.2)**. Two facts shape the whole document:
>
> 1. The prototype is a **local, single-user, terminal-based simulator** (Python + NumPy). It has **no web/API server, no database, no accounts, no network exposure and no real quantum hardware**. App-security controls that only exist in web deployments (auth, RBAC, TLS, CORS, CSRF, rate limiting, containers) are therefore marked **N/A (no such component)** rather than "Planned", and the conditions under which they would become relevant are stated.
> 2. Security *within the simulated protocol* — replay protection, authorisation, freshness, input validation, deterministic reproducible experiments and structured event logging — **is** implemented and verified, and this document claims only that.

---

## Project Details Used for This Document

| ITEM | CURRENT INFORMATION (VERIFIED) |
|---|---|
| Project | QuantumSigGuard — Quantum-Inspired Cyber Threat Detection for Digital Signature Security |
| SIH Problem Statement | SIH26141 |
| Organization | Egreen Quanta LLP |
| Theme | Security |
| Core focus | Teleportation-based Quantum Digital Signatures |
| Implementation language | Python 3.10+ (developed/verified on 3.14.7) |
| Runtime dependency | NumPy only (`numpy>=1.24`) |
| Detection approach | Two-layer statistical + Pauli-measurement threshold rules; **no AI/ML** |
| Quantum execution | Exact classical simulation (state vectors / density matrices, ≤ 3 qubits per register) |
| Real quantum hardware | **None — not used, not claimed** |
| Dataset | **No external dataset** — all observations simulation-generated from a seeded protocol |
| Core threat scenarios | Forgery, impersonation, replay, unauthorized verification, quantum-channel manipulation |
| Key operations | Bell-state/teleportation simulation, Pauli corrections Z^m1 X^m2, projective measurements, verification, attack analysis |
| Interface | Terminal CLI (`qsentinel-demo` / `python -m qsentinel.cli`) + JSON/Markdown reports |
| AI/ML | Excluded by requirement and by construction |

## Executive Summary

- The prototype demonstrates a **simulation-based Quantum Digital Signature security framework**: Bell-state entanglement, teleportation, Pauli corrections, projective measurement, signature verification, five attack scenarios and two deterministic threshold-based detectors. It uses **no AI/ML**, per SIH26141.
- Its primary security objective is to protect the **integrity and authenticity of digital signatures** against forgery, impersonation, replay, unauthorized verification and simulated quantum-channel manipulation — and to make each detection decision explainable by measurement statistics and fixed thresholds.
- The critical security boundary is **between the trusted protocol/detection logic and all untrusted inputs** (message/attack configuration). This is enforced structurally: configuration dataclasses validate every value at construction, are immutable at runtime (`frozen`), and are the single source of truth for all thresholds (`qsentinel/config.py`).
- The simulated protocol itself implements replay protection (nonce cache), an authorisation set for verification, coverage/completeness checks and structured per-transaction security-event export (`results/scenarios.json`) — all covered by a 73-test suite.
- Web-scale controls (authenticated RBAC, managed secrets, TLS, CORS/CSRF, rate limiting, hardened containers, SIEM/audit infrastructure, backups) do **not exist and are not claimed**, because the implementation has **no web surface**. They are listed as future hardening only if/when a service layer is introduced.
- The prototype must **not** be represented as a production quantum cryptographic service or as having been validated on real quantum hardware.

---

# 1. Overview and Scope

## 1.1 Purpose

This document describes the security architecture, access-control posture, threat model, data protection approach and current limitations of the QuantumSigGuard prototype for SIH 2026: a teleportation-based QDS threat-detection framework that performs quantum distribution simulation, teleportation, Pauli corrections, projective measurement and statistical/threshold-based threat classification with **no AI/ML**.

## 1.2 In Scope

- QDS simulation logic (`qsentinel/quantum`, `qsentinel/protocol`).
- Signing and verification operations (`sender`, `verifier`, `freshness`).
- Simulated quantum-state/measurement information.
- Attack-simulation inputs (`qsentinel/attacks`).
- Statistical + Pauli-measurement detection logic (`qsentinel/detection`).
- Verification thresholds and configuration (`qsentinel/config.py`).
- Protocol-level authorisation and replay controls.
- Security-event capture and export (`qsentinel/reporting`, `results/`).
- Source code, dependency and configuration integrity.
- Prototype (local CLI) deployment environment.

## 1.3 Out of Scope

- Real QKD infrastructure, physical quantum channels, entangled-photon generation or quantum processing units.
- Hardware Security Modules and production private-key management.
- Enterprise identity federation, certificate authority infrastructure or formal QDS certification.
- Production replacement of RSA/ECC signing systems.
- Web application / API security **beyond what exists** (there is no web surface today).
- Physical attacks on quantum hardware; nation-state-level certification; SLA/HA/DR guarantees.

---

# 2. Assets and Data Classification

| ASSET | CLASSIFICATION | SECURITY REASON |
|---|---|---|
| Public SIH problem statement | **Public** | Intended for challenge participants |
| Project description / README | **Public** | Suitable for SIH presentation |
| Architecture & math documentation | **Internal** | Reveals implementation structure |
| Application source code | **Internal** | Unauthorized modification could alter verification behaviour |
| Quantum/QDS simulation logic | **Internal** | Core implementation |
| Attack-simulation logic | **Internal** | Misuse could undermine demonstrations or testing |
| Verification thresholds & configuration | **Internal** | Unauthorized modification changes detection outcomes → **validated, immutable at runtime** |
| Simulated signing keys / transaction material | **Internal** | Integrity and impersonation risk **within the simulation**; never real keys |
| Measurement/statistical output | **Internal** | Contains security-test results |
| Security-event records (`results/*.json`) | **Confidential** | May reveal attack attempts / internal behaviour |
| Authentication secrets, API credentials | **N/A** | No accounts, credentials or API exist |
| Production signing keys | **Not claimed** | Would require HSM-grade key management |

**Key distinction (enforced in docs/`README`):** the simulated per-transaction key and signature material are *experiment data*, not real cryptographic key material. They are generated by seeded RNG, kept in memory/JSON for the session and never presented as production secrets.

---

# 3. Threat Model

## 3.1 Trust Boundaries

```
Operator (trusted local user)
   │  CLI arguments / config
   ▼
Input validation layer  (config __post_init__ + CLI range checks)
   │  validated ExperimentConfig (frozen)
   ▼
QDS Protocol Engine  (sender → channel → verifier)
   │
   ▼
Detection Engine  (statistical + Pauli layers)
   │
   ▼
Results / security-event export (JSON + Markdown)
```

Every externally-supplied value (message size, attack key, rates, trials, thresholds in config files) crosses the validation boundary before touching protocol logic. There is **no remote trust boundary** today — no network input path exists.

## 3.2 Likely Threat Actors

Within the simulator: forgers, impersonators, replay attackers, unauthorized verifiers and channel manipulators (each a modelled scenario). Around the tool itself: a local adversary who can modify source/config/files; and a compromised dependency or environment. Public API abusers are **not applicable** (no public interface).

## 3.3 STRIDE Threat Analysis (with implemented status)

| STRIDE | PROTOTYPE-SPECIFIC THREAT | EFFECT | STATUS / MITIGATION |
|---|---|---|---|
| **Spoofing** | Impersonating the signer/verifier in the protocol | Unauthorized signature acceptance | **Implemented** — impersonation scenario; identity bound to key via verification mismatch |
| **Spoofing** | Stealing application credentials | Unauthorized system access | **N/A** — no accounts/credentials exist |
| **Tampering** | Changing signature / channel data | Invalid signature accepted; manipulated measurements | **Implemented** — forgery + channel-manipulation scenarios and both detectors |
| **Tampering** | Modifying thresholds/config files | Distorted detection outcomes | **Partial** — validated, immutable-at-runtime config, single source of truth; no OS-level file ACLs (single-user local tool) |
| **Repudiation** | Denying an experiment/configuration run | Reduced accountability | **Implemented** — per-transaction security-event export with attack, nonce, signals, thresholds, verdicts |
| **Information Disclosure** | Leaking logs/simulation keys | Unauthorized use | **Partial** — logs are structured and local; no audit-store access control (single-user) |
| **DoS** | Excessive simulation/requests | CPU/memory exhaustion | **N/A** in CLI (bounded locally); rate limits **apply only if an API is added** |
| **Elevation of Privilege** | Normal user changes security settings | Security configuration altered | **N/A** — single-user CLI; no role hierarchy. Protocol-level authorisation is modelled in-simulation |

## 3.4 Project-Specific Security Risks

### Signature Forgery — implemented detection

An attacker signs with a guessed key (`guess_accuracy = 0.5`). Wrong guesses are eigenstates of the wrong Pauli operator; the verifier's own-basis measurement then returns a random ±1, so mismatch becomes visible. Measured: 100% detection by both layers (see `docs/results.md`).

### Impersonation — implemented detection

An attacker submits a *well-formed* signature bound to their own key and relabels the sender. Mismatch concentrates exactly on positions where the two keys differ. Measured: 100% detection.

### Replay Attack — implemented detection

The verifier keeps a `ReplayCache` of accepted nonces (`freshness.py`, cache size 512). A replay is rejected because `freshness_ok == False`, independently of the quantum data. Measured: 100% detection by the statistical layer (and correctly 0% by the measurement layer, which by design is blind to it — `docs/detection.md`).

### Unauthorized Verification — implemented detection

Verification is gated by an `authorised_entities` set; the verifier reports `unauthorised verifying entity`, and the statistical layer raises its `authorisation` signal. Measured: 100% detection.

### Quantum-Channel Manipulation — implemented detection

The channel-manipulation scenario injects Pauli errors on copies, forges classical teleportation bits (wrong Pauli correction → residual Pauli), drops copies (coverage shortfall) and disturbs the entanglement witness. Measured: 100% detection. The four disturbance channels are scored separately in `channel_manipulation.py` because they are *not* equally powerful (classical-bit tampering flips an eigenvalue only half the time).

### Dataset Tampering

**N/A / substitute control.** No external dataset is used. The equivalent risk — a modified evaluation producing misleading metrics — is addressed by **deterministic seeding** (one top-level seed drives all RNG), immutable config, and `results/config.json` recording the full configuration alongside every result, so any claimed result is reproducible byte-for-byte.

### Model Poisoning

**Not applicable** — no learned model. The equivalent concern is **algorithm/configuration poisoning** (thresholds, attack parameters, sample sizes). Mitigated by: thresholds fixed in `DetectionConfig` and never fitted; validation in `__post_init__`; and the test suite locking behaviour.

### Adversarial Inputs

Mitigated by **strict validation at every input boundary**: - `ProtocolConfig`, `ChannelConfig`, `DetectionConfig.__post_init__` reject out-of-range values (thresholds outside `[0,1]`, `parity_group_size > copies_per_bit`, `sign_bit ∉ {0,1}`, etc.). - CLI validates trial counts (`1..10000`) and group sizes (`≥ 1`); unknown scenarios raise `KeyError`; malformed numeric lists fail fast. - Attack constructors validate `guess_accuracy`, fractions etc. in `[0,1]`.

### API Abuse

**N/A today** — no API exists. If a web service is added later, authentication, authorization, rate limits and input-size limits must be introduced before exposure (see §13 roadmap).

---

# 4. Authentication

**No user authentication exists and none is needed in the current implementation** — the tool is a local, single-operator CLI. Any statement of authentication "TBD" is replaced by: **N/A (no accounts)**. If the prototype is later wrapped in a web service:

- authenticate before protected actions; Argon2id/bcrypt hashing; never store plaintext passwords;
- short-lived sessions/tokens that carry no secret QDS material;
- MFA for administrators; rate-limited logins; no username-enumeration oracle;
- keep any OAuth/OIDC credentials server-side and out of the repository.

## 4.1 Demo Credentials

**N/A** — no accounts. (`run_demo.py` is an unauthenticated local example; nothing is exposed remotely.)

---

# 5. Authorization and Access Control

## 5.1 In the implemented prototype

There is **no application role system** (single local operator). What *is* implemented is **protocol-level authorization**, modelled inside the simulator:

- `QDSSession` owns an `authorised_entities` set (default `{sender, verifier}`) and hands it to every attack context and the detection input (`session.py:91`, `build_input`).
- The `authorisation` statistic (`statistical.py:141`) and the verifier's `unauthorised verifying entity` reason (`verifier.py:241`) jointly reject transactions whose entity is not authorized.

| RESOURCE / ACTION (IN-SIMULATION) | AUTHORISED ENTITY | UNAUTHORISED ENTITY |
|---|---|---|
| Submit/verify a signed transaction | Allowed | Rejected (authorisation signal) |
| Reuse an accepted nonce | Rejected (replay signal) | Rejected |
| Run CLI `scenarios`/`evaluate` etc. | Local operator only | N/A (no remote access) |
| Modify thresholds/config | Single operator, file-level | N/A |

## 5.2 Recommended target matrix — only if a web/API layer is added

| RESOURCE / ACTION | ADMIN | ANALYST | DEMO/VIEWER |
|---|---|---|---|
| View results | ✓ | ✓ | ✓ |
| Run verification / approved scenarios | ✓ | ✓ | Optional (read-only preferred) |
| Modify thresholds | ✓ | ✗ | ✗ |
| View security-event logs | ✓ | Limited | ✗ |
| Manage datasets/config | ✓ | Limited | ✗ |
| Manage accounts/secrets | ✓ | ✗ | ✗ |

Least privilege: deny-by-default, separate users from configuration, prevent demo accounts from changing security settings. **Not implemented today because there are no such surfaces.**

# 6. Data Protection

## 6.1 Data in Transit

**N/A** — the tool never opens a socket; all data stays on the local host. If deployed as a service later: HTTPS with trusted certificates, redirect plain HTTP, do not directly expose backend ports.

## 6.2 Data at Rest

Persisted artifacts are plain JSON under `results/` (config, evaluation, sweeps, scenario events). They contain **no personal data and no secrets**. Best practice already followed:

- `results/config.json` records the full configuration with every result (reproducibility).
- Generated artifacts are reproducible from seed + code (`scripts/generate_report.py`).

If a database is introduced later: restricted credentials, non-public backups, OS-level protection for confidential logs.

### 6.3 Secrets Management

**There are no secrets.** The repo requires no API keys, tokens, credentials or environment variables (runtime dependency is NumPy only). Verified by a source scan for password/secret/credential/token/private-key patterns (§13.1). Therefore:

- No API keys in source code — **vacuously satisfied**.
- No credentials in `.git` history — **none exist** (see §13.1).
- `.env` / `.env.example` — **not needed**; a future service layer should add `.env.example` with variable names only.

### 6.4 Anonymization

**N/A** — no personal data is collected or stored.

### 6.5 Retention and Deletion

`results/` files are regenerable artifacts; deleting them is always safe (no state loss beyond regenerable reports). No formal retention policy exists; a service deployment should define one for audit records.

---

## 7. Application Security

### 7.1 Input Validation — Implemented

Verified controls (`qsentinel/config.py`, `qsentinel/cli.py`, attack constructors):

| INPUT | RULE | WHERE |
|---|---|---|
| Thresholds (mismatch, QBER, per-basis, distance, correlator, parity) | `0.0 <= v <= 1.0` | `DetectionConfig.__post_init__` |
| Decoh/tamper/loss rates | `0.0 <= v <= 1.0` | `ChannelConfig.__post_init__` |
| `message_length`, `copies_per_bit`, `witness_pairs`, `parity_group_size` | positive; `group <= copies`; `sign_bit ∈ {0,1}` | `ProtocolConfig.__post_init__` |
| `seed`, replay cache | `cache >= 1` | `DetectionConfig.__post_init__` |
| CLI trials | `1..10000` | `cli._parse_trials_safe` |
| CLI group sizes | positive integers | `cli.command_group_sweep` |
| Unknown scenario | `KeyError` | `session.run_scenario` |

Failures are loud (`ValueError` / `SystemExit`), never a silent change of algorithm behaviour.

### 7.2 Output Encoding / XSS

**N/A** — no browser rendering. Console output is plain text/tables; exported reports are JSON/Markdown containing only program-generated values.

### 7.3 File Upload Security

**N/A** — there are no file uploads.

### 7.4 SQL Injection

**N/A** — there is no database.

### 7.5 CORS / CSRF

**N/A** — no web server, no browsers, no cookies.

### 7.6 Rate Limiting

**N/A / Planned-future.** Local CLI runs are resource-bounded by the operator. If a web service is added, rate-limit login, verification, attack simulation and any expensive computation.

### 7.7 Dependency Security

**Partial.**

- Runtime dependency surface is minimal: **NumPy only** (`requirements.txt`: `numpy>=1.24`; `pyproject.toml` same).
- Pinning: a version *floor* is declared, but there is **no lockfile** and **no automated vulnerability scanner**. Pin/lock and scan before any wider distribution.

---

## 8. Model and Algorithm Security

### 8.1 No AI/ML Classification — Implemented by design

There is no trained model, no weights, no training/inference split, and no learned threshold. Every detection decision is a deterministic comparison of measured protocol statistics against fixed, documented thresholds (`docs/math.md`, `docs/detection.md`). This satisfies the SIH constraint that detection use quantum/statistical principles rather than AI/ML.

### 8.2 Quantum-Inspired vs Real Quantum Hardware — honest boundary

The correct description, used throughout the repo docs:

> A classical software simulation / quantum-inspired implementation of teleportation-based QDS security concepts.

It does **not** claim real quantum channels, entangled-photon transmission, production QKD, hardware-level information-theoretic security, or any quantum computational advantage.

### 8.3 Algorithm Integrity — Implemented mitigations, no cryptographic release signing

Integrity-sensitive components (protocol, basis operations, Pauli corrections, verification logic, thresholds, attack parameters) are protected by:

- **Single source of truth** for every threshold (`qsentinel/config.py`; PRD requires no duplication) — `docs/prd-traceability.md`;
- **Immutable-at-runtime** frozen config dataclasses;
- a **73-test suite** (`python3 -m unittest discover -s tests`) that locks expected behaviour, including the Pauli-correction table, teleportation fidelity ≈ 1, Bell correlators and every detector signal;
- **deterministic reproducibility** via a single top-level seed and `results/config.json`.

Not present (future work): cryptographic release hashes / code signing.

### 8.4 Data / Dataset Validation

**N/A with substitute.** No external dataset. "Validation" is: seeded, reproducible generation; config recorded beside results; attack/scenario keys validated; no hidden randomness.

### 8.5 Reproducibility — Implemented

Every experiment records: seed, protocol/channel/detection configuration, attack type, trials, thresholds, per-signal values, verdicts and runtime — i.e. `results/config.json`, `results/evaluation.json`, `results/scenarios.json`. Rerunning with the same seed reproduces the report (verified during development).

---

## 9. Infrastructure and Deployment Security

### 9.1 Local Demo — current deployment

- The tool binds **nothing** to a network; it runs as a normal local process (no debug ports, no listeners).
- Runs under the invoking user's account; no privileged operations are performed.
- No `.env`, no confidential configuration, no network services to lock down.
- Console errors are the CLI's own messages; there is no remote-facing debug mode.

### 9.2 Container Hardening

**N/A today** — not containerized. If containerized later: minimal base image, non-root user, pinned image tags, no secrets in the image, CPU/memory limits.

### 9.3 Environment Separation

**N/A today** — no environments. A future service should keep dev/demo/prod separate and never share secrets between them.

### 9.4 Network Exposure

**N/A today** — nothing listens on a network. No firewall changes, no open ports.

### 9.5 Backups

**N/A / trivial.** All state is regenerable (code + seed). No database or user data exists.

---

## 10. Logging, Monitoring and Incident Response

### 10.1 Security-Event Logging — Implemented

The SIH expected solution explicitly includes security-event logging, and it is present: `ScenarioResult.as_dict()` (`session.py:58`) exports, per transaction:

- scenario + attack title and `is_attack`/notes;
- verification outcome: `signature_valid`, `freshness_ok`, `authorisation_ok`, `reject_reason`, `mismatch_count`, `check_count`, `mismatch_rate`, failing checks;
- both detector verdicts and **every signal with its value, threshold, comparison and tripped state** (`detection_measures`);
- `elapsed_seconds` for benchmarking.

The `scenarios` CLI command and `scripts/generate_report.py` write this to `results/scenarios.json` (and `scenarios.md`). This is a structured, per-event log — not yet a tamper-evident / append-only audit store or a SIEM feed.

### 10.2 Information That Must Not Be Logged

**Policy, vacuously satisfied:** no passwords, tokens, API secrets or private signing keys exist to log. Simulated per-transaction keys are experiment material, not credentials, and are never exported into reports.

### 10.3 Monitoring / Alerting

**N/A today** (no service, no streams). For a future deployment, alert on repeated verification failures, malformed input bursts, unauthorized admin attempts and configuration changes.

### 10.4 Prototype Incident-Response Procedure

For the local tool:

1. **Detect** — abnormal verdicts, failing tests, or anomalous files in `results/`.
2. **Contain** — stop running the tool; quarantine the environment.
3. **Preserve evidence** — keep `results/`, config and environment details.
4. **Investigate** — is the anomaly a modelled attack, an invalid input, a bug, or a tampered config/code?
5. **Remediate** — fix the defect; revert any modified code/config; rotate **nothing** (no secrets).
6. **Validate** — rerun the 73-test suite and the seeded report generator.
7. **Document** — record cause, impact, fix, prevention.

---

# 11. Privacy and Compliance Alignment

Alignment references only; **no certification or legal-compliance claim** is made.

## 11.1 Digital Personal Data Protection Act, 2023

**Low direct applicability** — the prototype processes no personal data (all data is simulated). If identifiable data is ever introduced, apply purpose limitation, protection, controlled retention, secure deletion and unauthorized-access controls.

## 11.2 Information Technology Act, 2000

General practice applies to any future public deployment (authorized access only, adequate security measures). No network-facing component exists today.

## 11.3 OWASP

| OWASP AREA | STATUS IN THIS PROTOTYPE |
|---|---|
| Broken Access Control | Protocol-level authorisation implemented; app-level RBAC N/A (no accounts) |
| Cryptographic Failures | No real crypto/keys; simulation keys are ephemeral experiment data |
| Injection | No interpreter/SQL/HTML inputs to inject; config validated |
| Insecure Design | Threat model + trust boundaries documented here and in `docs/architecture.md` |
| Security Misconfiguration | Single source of truth, validated, immutable config; no remote debug/dev surface |
| Vulnerable Components | NumPy version floor declared; **no lockfile/scanner yet** (future) |
| Authentication Failures | N/A (no authentication) |
| Integrity Failures | Seeds + config-record + 73-test suite protect result/tool integrity |
| Logging Failures | Structured security events exported (see §10.1) |
| SSRF | N/A (no URL-fetching code) |

## 11.4 NIST CSF Alignment

- **Identify** — assets (§2), protocol components, threat model (§3).
- **Protect** — validation (§7.1), reproducible config, authorisation + replay controls (§5).
- **Detect** — the two detectors, freshness/authorisation rules, security-event export (§10.1).
- **Respond** — local incident procedure (§10.4).
- **Recover** — regenerate `results/` from code + seed; rerun tests.

This is an alignment mapping, not an attestation.

---

# 12. Known Limitations and Future Hardening

### 12.1 Current Limitations

- Quantum behaviour is **simulated**, not measured on real hardware; physical noise, decoherence timing and hardware imperfections are idealized (Pauli/tamper/loss model — `docs/limitations.md`).
- **No authentication, accounts, RBAC, MFA, TLS, rate limiting or monitoring** exists — because there is no web service; these become required only after a service layer is added.
- No **tamper-evident/append-only audit store**; events are structured JSON, not WORM logs.
- Config/threshold file edits are not blocked by OS-level ACLs (a local single-user tool).
- No **dependency lockfile or vulnerability scanner**.
- No cryptographic **release hashing/code signing** for the tool itself.
- No external/public dataset; results are simulator-generated (by design).
- The measurement-based (Pauli) layer **cannot** detect replay or unauthorized verification — this is a *documented protocol property*, reported honestly as measured false negatives (`docs/detection.md`, `docs/results.md`) rather than hidden.

### 12.2 Prioritized Hardening Roadmap

| PRIORITY | IMPROVEMENT | STATUS TODAY |
|---|---|---|
| **P0** | Strict input/range validation | **Done** (§7.1) |
| **P0** | Single, immutable threshold configuration | **Done** (§8.3) |
| **P0** | Protocol-level replay + authorisation controls | **Done** (§3.4) |
| **P0** | Structured security-event export | **Done** (§10.1) |
| **P0** | Reproducible seeded experiments | **Done** (§8.5) |
| **P1** | Dependency lockfile + vulnerability scanning | Planned |
| **P1** | Release integrity hashes / signed artifacts | Planned |
| **P1** | Tamper-evident audit store (only if service layer) | Planned |
| **P2** | Authentication + RBAC + rate limiting (only if web API) | Planned-on-adoption |
| **P2** | HTTPS/TLS, hardened containers, monitoring (only if web API) | Planned-on-adoption |
| **P3** | HSM/KMS-backed key management | Not applicable in simulation; only for a real deployment |
| **P3** | Real quantum-backend validation | Future research |

## 13. Security Checklist and Evidence

### 13.1 Checklist

| CONTROL | STATUS | EVIDENCE |
|---|---|---|
| Teleportation-based QDS concept | **Implemented** | `qsentinel/protocol/teleportation.py` |
| Bell-state entanglement | **Implemented** | `qsentinel/quantum/states.py`, `gates.py` |
| Pauli correction / measurement | **Implemented** | `teleportation.py` (`_CORRECTION_TABLE`, `Z^m1 X^m2`) |
| Statistical threshold detection | **Implemented** | `qsentinel/detection/statistical.py` |
| Pauli-measurement detection | **Implemented** | `qsentinel/detection/pauli.py` |
| AI/ML excluded | **Implemented** | entire `qsentinel/detection` (fixed rules); `docs/architecture.md` |
| Forgery scenario | **Implemented** | `qsentinel/attacks/forgery.py`, `tests/test_attacks.py` |
| Impersonation scenario | **Implemented** | `qsentinel/attacks/impersonation.py` |
| Replay scenario + freshness cache | **Implemented** | `qsentinel/attacks/replay.py`, `protocol/freshness.py` |
| Unauthorized-verification scenario | **Implemented** | `qsentinel/attacks/unauthorized.py`, `session.authorised_entities` |
| Channel-manipulation scenario (4 disturbance channels) | **Implemented** | `qsentinel/attacks/channel_manipulation.py`, `channel.py` |
| Security-event logging | **Implemented** (JSON events; not a WORM audit store) | `session.ScenarioResult.as_dict`, `reporting/export.write_json`, `results/scenarios.json` |
| Input validation (config/CLI/attacks) | **Implemented** | `config.py` `__post_init__`, `cli._parse_trials_safe` |
| Deterministic reproducibility | **Implemented** | one seed; `results/config.json` |
| No secrets / credentials anywhere | **Verified** | source scan (below); NumPy-only runtime |
| Dataset / dataset checksum | **N/A** (no external dataset; seeded simulation instead) | `results/config.json` |
| Authentication | **N/A** (no accounts) | — |
| MFA / RBAC / admin roles | **N/A** (no accounts); protocol authorisation implemented | `session.py:91`, `verifier.py:241` |
| HTTPS/TLS, CORS, CSRF, rate limiting | **N/A** (no web server) | — |
| File uploads / SQL injection | **N/A** (no uploads, no database) | — |

| CONTROL | STATUS | EVIDENCE |
|---|---|---|
| Dependency pinning | **Partial** (floor only, no lockfile) | `requirements.txt`, `pyproject.toml` |
| Dependency vulnerability scanning | **Planned** | — |
| Containers / backups / monitoring | **N/A today** (local CLI; regenerable state) | — |
| Incident-response procedure documented | **Implemented (documentation)** | §10.4 |
| Real-quantum / production-CSR claims | **Explicitly not claimed** | §8.2, `docs/limitations.md` |

### 13.2 Evidence Map (file → control)

| CONTROL | FILE(S) |
|---|---|
| Threshold & config source of truth | `qsentinel/config.py` |
| Validation invariants | `qsentinel/config.py` (`__post_init__` × 3) |
| Immutable config | `frozen=True` dataclasses, `config.py` |
| Replay cache | `qsentinel/protocol/freshness.py` |
| Authorisation set | `qsentinel/session.py:91` |
| Authorisation/replay signals | `qsentinel/detection/statistical.py:133–148` |
| Reject reasons | `qsentinel/protocol/verifier.py:228–256` |
| Event record | `qsentinel/session.py:58–73` |
| Event export | `qsentinel/reporting/export.py`, `scripts/generate_report.py` |
| No-secrets scan | `grep -rn "password\|secret\|token\|credential\|private key" qsentinel/` → **no matches** (only doc strings using the word "secret" for the simulated key concept) |
| Behaviour lock | `tests/` (73 tests) |
| Reproducibility | one seed (`config.DEFAULT_SEED`), `results/config.json` |

A scan for credential patterns returns no matches: the matched strings in the repository are docstring uses of the word "secret" describing the *simulated* signing key, not stored credentials.

---

# Final Security Position

QuantumSigGuard is a **research and demonstration security framework for teleportation-based QDS threat detection**. It applies quantum-inspired protocol operations and deterministic statistical thresholding to detect forgery, impersonation, replay, unauthorized verification and channel manipulation, with **no AI/ML** — and it closes the loop with structured, reproducible security-event output and behaviour-locking tests.

Its principal strengths for the SIH prototype stage are: detection decisions are **explainable** down to individual measurement statistics and thresholds (§8.1); results are **reproducible** from a single seed (§8.5); inputs are **validated** at every boundary (§7.1); the simulated protocol itself enforces **replay protection and authorisation** (§5.1); and the project states its limits honestly (simulation-only, no real quantum hardware, measurement-blind spots on replay/unauthorized — §12.1).

Simulation security and production application security remain separate concerns. Because the current artifact has **no web surface, no accounts, no secrets and no network exposure**, the web-era controls are marked **N/A** instead of being overclaimed as "planned"; they become requirements only when a service layer is introduced. This distinction keeps the submission technically credible: it demonstrates the required **QDS security and attack-detection concept** without overstating production or real-quantum security guarantees.

# 10. Limitations and Honest Scope

This prototype is a **pedagogical, classical-NumPy simulation** of a quantum-inspired detection pipeline. It is not a production security product and its numbers should be read as mean-behaviour demonstrations, not as physical guarantees.

## Conceptual limits

1. **Classical simulation, not real quantum hardware.** All states are dense amplitude vectors and all outcomes are sampled from exact Born-rule probabilities. Real devices introduce continuous gate errors, finite visibility, measurement misclassification and decoherence profiles that a bit-flip Pauli model does not capture.
2. `reduced_single_qubit` **uses the dominant eigenvector.** For a single-qubit *product* state (every state this protocol measures) the marginal is pure and the eigenvector reproduces the exact state and all Pauli expectations. It is deliberately **not** a general mixed-state partial trace; the Bell-pair marginal in the entanglement witness is measured with the exact two-body correlator, not with this helper.
3. **Protocol scale.** `L = 8` and `n = 4` keep the simulation fast; the statistical power of the thresholds (e.g. `QBER > 0.02`) and the noise "knee" at `p ≈ 0.02–0.05` are consequences of that small register, not of the protocol design at large `L`. Scaling `L` and `n` requires re-deriving operating points, not just re-running.
4. **No AI/ML.** The two detectors are fixed threshold rules. When the problem statement asks for a "quantum-inspired model", this prototype answers with closed-form Pauli statistics plus classical protocol rules. It deliberately uses **no neural network, no statistical classifier**; nothing is trained or fitted to the attack data.

## Detection limits the prototype measures

5. **Replay and unauthorised verification are quantum-invisible.** Measurement statistics cannot distinguish a replayed transcript from a fresh one. Catching them requires the classical freshness (nonce cache) and authorisation rules, which are part of the statistical layer. The evaluation therefore reports the Pauli layer at 60% detection by design.
6. **Threshold calibration is hand-set.** The thresholds in `DetectionConfig` are analytic choices targeting low false alarms for `p ≤ 0.01`. A real deployment should calibrate them against the actual hardware error budget.

## Threat-model limits

7. The simulator assumes an attack on the channel, the signature, or the verification *context*. It does **not** model a malicious verifier, a corrupted shared-key infrastructure, a denial of service at the transport layer, or side channels (timing, EM, power) on the simulated hardware.
8. `pauli_decoherence` is a depolarising Pauli channel with uniform weights. Real amplitude/phase damping, correlated errors and coherent (unitary) errors are not modelled, although the same verification statistics would partially respond to them.

## Engineering scope

9. Performance: the statevector engine is exact but unoptimised; the per-copy teleportation allocates intermediate arrays. Runtime is ~11 ms per transaction on this machine, fine for a prototype and a SIH demo, not for a high-throughput service.
10. Imports/dependencies: the package requires only NumPy and runs on CPython ≥ 3.10 (developed against 3.14.7/NumPy 2.5.x).

A full threat-and-limitations table mapping to the PRD is in `docs/prd-traceability.md`.

# 11. PRD Traceability

Mapping between the problem statement's functional requirements and the implementation. The identifiers (FR-*) are the sections of the SIH 2026 problem brief for "Quantum-Inspired Cyber Threat Detection for Digital Signature Security".

| REQUIREMENT | IMPLEMENTATION | STATUS |
|---|---|---|
| FR-01 Understand and simulate the Quantum Digital Signature flow | `qsentinel/protocol/`, `qsentinel/protocol/teleportation.py`, `docs/math.md` | Implemented |
| FR-02 Understand and simulate quantum teleportation | `qsentinel/protocol/teleportation.py` (exact 3-qubit circuit, Pauli correction), `tests/test_teleportation.py` | Implemented |
| FR-03 Build a quantum/classical simulation of the QDS signature and verification flow | `qsentinel/session.py`, `qsentinel/channel.py`, `scripts/generate_report.py` | Implemented |
| FR-04 Simulate and implement quantum attacks | `qsentinel/attacks/` (forgery, impersonation, replay, unauthorized, channel), `docs/attacks.md` | Implemented |
| FR-05 Detect quantum attacks using a quantum-inspired detection model | Two non-AI layers: `qsentinel/detection/statistical.py` and `pauli.py`; `DetectionEngine` combines them | Implemented |
| FR-06 Model and simulate a quantum attack dataset | `qsentinel/attacks/` + `QuantumChannel` noise model; labelled trials in `qsentinel/evaluation/runner.py` | Implemented |
| FR-07 Evaluate the model; report detection rate and accuracy | `qsentinel/evaluation/{metrics,runner}.py`, `docs/results.md`, `results/evaluation.json` | Implemented |
| FR-08 (Submission) architecture, math and results documentation | `docs/architecture.md`, `docs/math.md`, `docs/results.md` | Implemented |
| FR-09 (Submission) runnable demo | `run_demo.py`, `qsentinel/cli.py`, `scripts/` | Implemented |
| FR-10 (Submission) working prototype via UI/CLI | `qsentinel/cli.py` (`scenarios`, `demo`, `evaluate`, `noise-sweep`, `group-sweep`); console tables + JSON/Markdown export | Implemented |
| FR-11 No AI/ML is required for detection | Both layers are fixed arithmetic/classical rules; nothing is trained or fitted | Implemented |
| FR-12 Produce quantitative results: detection rate, false-alarm rate, accuracy | `ConfusionMatrix` + Wilson 95% intervals in `qsentinel/evaluation/metrics.py` | Implemented |
| FR-13 Quantum measurement statistics are computed from the circuit, not faked | Every statistic flows through `measure_pauli` / `parity_measurement` / `correlator` over exact state vectors | Implemented |
| FR-14 Pre-defined thresholds on measurement data | `DetectionConfig` (single source, documented defaults) | Implemented |
| FR-15 Measure outcomes cover the full range of measurement statistics | eigenvalue agreement, basis distributions, cat-state parity, two-body correlators | Implemented |
| FR-16 Compare detection methods / baselines | `DetectionEngine` runs both layers on the shared `DetectionInput`; `docs/detection.md` | Implemented |
| FR-17 Run evaluation over noise and amplification settings | `run_noise_sweep`, `run_group_size_sweep`, `results/noise_sweep.json`, `results/group_sweep.json` | Implemented |
| FR-18 Deliverables: code, docs, results, demo, README | Complete repository under `~/Desktop/QuantumSentinel` | Implemented |

## Threat-model coverage table

| PRD THREAT | ATTACK MODULE | DETECTED BY | NOTES |
|---|---|---|---|
| Signature forgery | `SignatureForgery` | statistical + Pauli | guessed-key mismatch in measurement data |
| Sender impersonation | `Impersonation` | statistical + Pauli | well-formed signature bound to wrong key |
| Replay | `ReplayAttack` | statistical only | quantum-invisible; nonce cache |
| Unauthorised verification | `UnauthorizedVerification` | statistical only | quantum-invisible; authorisation rule |
| Quantum channel manipulation | `ChannelManipulation` | statistical + Pauli | Pauli/tamper/loss + witness disturbance |

# 12. Feature Backlog

This is the feature ticket list, corrected so that every ticket's status reflects the actual repository rather than the template's "mostly Not Started" state. Statuses were assigned by verifying the named code below, not by intent. Where a feature exists only in CLI/console form or as the optional local web dashboard (see the `[web]` extra and `streamlit_app.py`), that is stated explicitly.

**Facts used throughout.** Repo: `qsentinel/`, Python 3.14.7, numpy 2.5.3. No web/API server, no database, no Docker, no Qiskit/real hardware, no AI/ML learners. The "backend/frontend" boundary is `QDSSession` + CLI (`qsentinel/cli.py`) + reporting (`qsentinel/reporting/`), plus an optional read-only Streamlit view layer (`ui/`, `app_pages/`, `web_helpers.py`). Measured results were produced with seed `20260101`, 200 trials/attack (1200 total): statistical detector 1.000 [0.996,1.000]/FAR 0.000; Pauli 0.600 [0.569,0.630]/FAR 0.000; ~11 ms/transaction; 73 tests green.

---

## 1. Account in this backlog

```
# Backlog Team:QuantumSentinel Project:SQDS Type:feature Rank:CSV-id
## Feature Backlog (CSV export) + GUI panel
This backlog owns the QDS prototype capability (SIH-25182): quantum core, teleportation,
threat simulation, detection, evaluation, CLI/console interface and documentation.
```

## 2. Correct backlog assumptions

Replace the template's placeholder rows with the verified stack:

| FIELD | WAS (TEMPLATE) | CORRECT (REPO) |
|---|---|---|
| Backend runtime | web service | single-process Python package (`qsentinel/`) |
| Backend interface | REST/HTTP | CLI commands (`demo`, `scenarios`, `evaluate`, `noise-sweep`, `group-sweep`) via `QDSSession` |
| Data storage | database | JSON files under `results/` (`write_json`, `generate_report.py`) |
| Data generation | 20,000 tx | 1200-trial evaluation, 200/attack, seed `20260101` (deterministic) |
| Frontend | browser screens | ANSI console tables/bars + JSON/Markdown export (`docs/frontend.md`) |
| Quantum backend | real QPU | exact state-vector simulation (numpy); no real-hardware claims |
| AI/ML | model | none — fixed-rule detectors only |

## 3. Corrected ticket list

Status shorthand: **Done** (verified in code), **Partial** (some acceptance criteria only), **N/A** (not applicable to a CLI-only synchronous product), **Not Started** / **Won't** (kept honestly).

### EP-01 Core QDS protocol

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-001 | Deterministic simulation configuration | feature | **Done** | 1 SP | Must | deterministic, configurable |
| FT-002 | Bell-state generation and entanglement | feature | **Done** | 2 SP | Must | quantum |
| FT-003 | Quantum teleportation of a message state | feature | **Done** | 2 SP | Must | quantum |
| FT-004 | Pauli correction logic | feature | **Done** | 1 SP | Must | core |
| FT-005 | Projective verification (final measurement) | feature | **Done** | 2 SP | Must | quantum |

- **FT-001** `config.py` frozen dataclasses; seed `20260101`; every run reproducible; frozen config emitted to `results/config.json`. Validation in `__post_init__`.
- **FT-002** `quantum/states.py`, `quantum/gates.py` — exact state-vector Bell pair `(|00>+λ|11>)/√2`.
- **FT-003** `protocol/teleportation.py` — Bell, measurement, classical bits, reconstruction `Z^m1 X^m2`.
- **FT-004** `_CORRECTION_TABLE`, `(1,1)→Y (≡ ZX up to global phase)`.
- **FT-005** `protocol/verifier.py` — projective measurement in `v` basis; parities + witnesses.

### EP-02 Threat simulation

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-006 | Simulate forgery (fundamental) | feature | **Done** | 2 SP | Must | attack |
| FT-007 | Simulate impersonation | feature | **Done** | 2 SP | Must | attack |
| FT-008 | Simulate replay | feature | **Done** | 2 SP | Must | attack |
| FT-009 | Simulate unauthorised verification | feature | **Done** | 1 SP | Must | attack |
| FT-010 | Simulate channel manipulation | feature | **Done** | 2 SP | Must | attack |

- All in `qsentinel/attacks/` with shared `Attack` base + registry; `ATTACK_ORDER` drives evaluation. **FT-010** also models four disturbance channels (`qsentinel/channel.py`).
- Measured 200/200 detection of forgery/impersonation/channel manipulation; replay & unauthorised quantum-invisible → caught only by classical rules (`docs/results.md`, `docs/detection.md`).

### EP-03 Detection engine

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-011 | Mismatch statistics (cryptographic) | feature | **Done** | 2 SP | Must | non-AI |
| FT-012 | Configurable thresholds | feature | **Done** | 1 SP | Must | configurable |
| FT-013 | Outcome classification | feature | **Done** | 2 SP | Must | core |
| FT-014 | Explain threat-detection evidence | feature | **Done** | 2 SP | Must | evidence |

- **FT-011/012** `detection/statistical.py`; thresholds single-sourced in `DetectionConfig` (mismatch 0.05, qber 0.02, per-basis 0.05, distribution-distance 0.20, correlator/parity 0.90, replay cache 512).
- **FT-013** `DetectionEngine` + `DetectionResult.verdict` (`THREAT`/`LEGITIMATE`, `ACCEPT`/`REJECT`).
- **FT-014** per-signal tables in `reporting/render.py` show rule + measured value + threshold + tripped state; no AI claim made anywhere.

### EP-04 Data & experiment pipeline

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-015 | Experiment result schema | feature | **Done** | 2 SP | Must | schema |
| FT-016 | Save experiment results | feature | **Done** | 2 SP | Must | persistence |
| FT-017 | Export results | feature | **Done** | 1 SP | Must | export |
| FT-018 | Validate experiment inputs | feature | **Done** | 1 SP | Must | validation |

- **FT-015** `TrialRecord`, `EvaluationResult.summary()`, `ScenarioResult.as_dict` (typed dataclasses, no fabricated values).
- **FT-016/017** JSON on disk (`results/*.json`), `reporting/export.py` (`write_json`, `scenario_to_markdown`), `scripts/generate_report.py`. No database — documented as by-design (storage = results bundle).
- **FT-018** `config.py __post_init__` (3 configs), CLI range checks, unknown attack → `KeyError`; negative tests in `test_config.py`, `test_cli.py`.

### EP-05 Backend / service layer

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-019 | Protocol execution service | feature | **Done (in-process)** | 2 SP | Must | service |
| FT-020 | Attack execution service | feature | **Done (in-process)** | 2 SP | Must | service |
| FT-021 | Detection integration | feature | **Done** | 1 SP | Must | integration |
| FT-022 | Backend error conventions | feature | **Done (CLI-level)** | 1 SP | Should | errors |

- **FT-019/020** the "service boundary" is `QDSSession` + CLI subcommands; no HTTP API exists (documented in `docs/frontend.md`). A web service wrapper would be new work.
- **FT-021** every scenario runs `DetectionEngine` before returning; verdict in the result.
- **FT-022** errors are surfaced as `ValueError`/`KeyError`/`argparse SystemExit` with field-specific messages; no tracebacks in normal flows.

### EP-06 Frontend / interface

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-023 | Experiment configuration screen | feature | **Done (CLI)** | 2 SP | Must | ui, cli |
| FT-024 | QDS protocol workflow view | feature | **Done (CLI)** | 2 SP | Must | ui, cli |
| FT-025 | Measurement-statistics visualisation | feature | **Done (console)** | 2 SP | Must | ui, cli |
| FT-026 | Threat verdict panel | feature | **Done (console)** | 2 SP | Must | ui, cli |
| FT-027 | Loading / empty states | feature | **N/A** | 1 SP | Should | ui |
| FT-028 | Actionable frontend errors | feature | **Done (CLI)** | 1 SP | Should | ui, cli |

- The "frontend" is the console/reporting layer (`docs/frontend.md`). **FT-023** CLI flags + presets (`clean` / `noisy` / `degraded`); **FT-024** protocol-stage blocks in `render_scenario`; **FT-025** per-basis measurement + signal tables; **FT-026** verdict banner with supporting values + thresholds; **FT-028** specific messages like `error: expected a positive integer, got 'x'`.
- **FT-027** N/A: no async loading (synchronous CLI); empty states render `(no rows)`; browser screens, charts and duplicate-submission prevention are future web work, not applicable to a CLI.

### EP-07 Evaluation & benchmarking

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-029 | Evaluation experiment matrix | feature | **Done** | 2 SP | Must | evaluation |
| FT-030 | Benchmark vs classical baseline | feature | **Partial** | 2 SP | Must | comparison |
| FT-031 | Attack-detection performance | feature | **Done** | 2 SP | Must | metrics |
| FT-032 | Computational performance | feature | **Done** | 1 SP | Should | performance |
| FT-033 | Benchmark summary view | feature | **Done (console)** | 1 SP | Should | ui, cli |

- **FT-029** `evaluation/runner.py run_evaluation` — benign + 5 attacks, configurable trials, no pre-filled results.
- **FT-030 Partial:** the statistical layer is documented as the deterministic baseline comparator and results avoid any superiority claim, but there is no separate standalone "classical baseline algorithm" to benchmark against (`docs/results.md` §10.5 discusses two baseline concepts). Honest gap.
- **FT-031** per-attack detection/FAR/accuracy + 95% Wilson CI for both detectors (`evaluation/metrics.py`, `results/evaluation.json`).
- **FT-032** `scripts/benchmark.py` wall-clock mean ~11 ms/tx over all scenarios; `elapsed_seconds` per run.
- **FT-033** console summary tables + JSON export (`render_evaluation`, `render_evaluation_table`); web charts N/A.

### EP-08 Security & reliability

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-034 | Log security-relevant events | feature | **Done** | 2 SP | Must | security |
| FT-035 | Sanitise untrusted inputs | feature | **Done** | 1 SP | Must | security |
| FT-036 | Handle execution failures | feature | **Partial** | 1 SP | Should | reliability |

- **FT-034** `ScenarioResult.as_dict` event records written to `results/scenarios.json` (attack type + verdict; no sensitive data logged) — verified by repo credential scan.
- **FT-035** config `__post_init__` + CLI range validation; zero untrusted-input paths (no remote input at all).
- **FT-036 Partial:** validated inputs prevent most failures and `KeyboardInterrupt→exit 130`; but there is no try/except wrapper that records a structured error record and lets a batch run continue after a mid-run failure.

### EP-09 Testing & deployment

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-037 | Protocol unit tests | feature | **Done** | 2 SP | Must | testing |
| FT-038 | Attack/detector tests | feature | **Done** | 2 SP | Must | testing |
| FT-039 | End-to-end integration test | feature | **Done** | 2 SP | Must | testing |
| FT-040 | Reproducible local deployment | feature | **Done** | 2 SP | Must | packaging |
| FT-041 | Offline demo fallback | feature | **Done** | 2 SP | Must | demo |

- **FT-037** `tests/test_quantum.py`, `test_teleportation.py`, `test_protocol.py`.
- **FT-038** `test_attacks.py`, `test_detection.py` (incl. threshold behaviour) + legitimate-case coverage.
- **FT-039** `test_cli.py` drives `scenarios`/`demo`/`evaluate`; all five attack paths exercised; **73 tests green** (`python3 -m unittest discover -s tests`). Retry-path verification N/A offline.
- **FT-040** `pyproject.toml` + `requirements.txt`; editable install verified in a clean venv.
- **FT-041** pre-generated `results/` (seed `20260101`) + `run_demo.py`; fully offline.

### EP-10 Documentation

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-042 | Document the math model | task | **Done** | 2 SP | Must | docs |
| FT-043 | Document the threat model | task | **Done** | 1 SP | Must | docs |
| FT-044 | README | task | **Done** | 1 SP | Must | docs |
| FT-045 | SIH demo script / walkthrough | task | **Done** | 1 SP | Must | demo |

- **FT-042** `docs/math.md`; **FT-043** `docs/attacks.md` + `docs/security.md`; **FT-044** `README.md`; **FT-045** three-minute demo path in `docs/frontend.md` (§2.3/§4.3) + `run_demo.py`.

### EP-11 Stretch (future)

| ID | TITLE | TYPE | STATE | ESTIMATE | PRIORITY | LABELS |
|---|---|---|---|---|---|---|
| FT-046 | Batch experiment runner | feature | **Done (absorbed)** | 3 SP | Could | script |
| FT-047 | Configurable noise models | feature | **Done (already in MVP)** | 3 SP | Could | quantum |
| FT-048 | Multi-threshold comparison | feature | **Partial** | 3 SP | Could | evaluation |
| FT-049 | Experiment history dashboard | feature | **Partial (read-only viewer)** | 3 SP | Could | web |
| FT-050 | Downloadable security report | feature | **Done (export)** | 2 SP | Could | export |
| FT-051 | Real quantum backend | feature | **Not Started (out of scope)** | 13 SP | Could | quantum |
| FT-052 | PostgreSQL persistence | feature | **Not Started (out of scope)** | 5 SP | Could | backend |
| FT-053 | Auth / access control | feature | **Not Started** | 5 SP | Could | security |
| FT-054 | Containerisation / deployment | task | **Not Started** | 3 SP | Could | devops |
| FT-055 | CI pipeline | task | **Not Started** | 2 SP | Could | devops |

- **FT-046 Done:** `scripts/run_evaluation.py`, `scripts/generate_report.py`, `evaluate --trials`, sweeps.
- **FT-047 Done:** `ChannelConfig(decoherence_rate, message_tamper_rate, loss_rate)` + presets `clean` / `noisy`(0.01)/ `degraded`(0.05) + `noise-sweep`.
- **FT-048 Partial:** thresholds are configurable and validated, but there is no automated threshold-sensitivity sweep (easy follow-up).
- **FT-049 Partial:** the optional read-only Streamlit viewer (`streamlit_app.py` + `app_pages/` + `ui/`) renders the results bundle and live runs, but there is no persisted per-run experiment history.
- **FT-050 Done:** report bundle = `results/{config, evaluation, scenarios, noise_sweep, group_sweep}.json` + `scenarios.md`.
- **FT-051/052** depend on hardware/database infrastructure this prototype deliberately excludes (`docs/limitations.md`); kept honest.
- **FT-053** protocol-level authorisation exists (`session.py`, docs/security.md §5); no user accounts yet.
- **FT-055** tests run via the documented unittest command; no CI wiring.

## 4. Corrected dependency graph

Effectively complete: EP-01 → EP-02 → EP-03 → EP-04 → EP-05 holds; EP-06 is satisfied by the CLI/reporting layer; EP-07→EP-10 are done. Remaining owners of open edges: FT-030, FT-036, FT-048 (Partial), FT-049 (Partial — read-only viewer), FT-053/054/055 (Not Started, stretch).

## 5. Corrected MVP scope

The MVP is **delivered**: EP-01..EP-04, EP-05(CLI form), EP-06(CLI form), EP-07, EP-08 (except FT-036+), EP-09, EP-10. Only the explicitly marked Partial/Not-Started/web items remain, and none blocks the demonstrable SIH deliverable.

## 6. Corrected risk list

| RISK | LIKELIHOOD | IMPACT | STATUS |
|---|---|---|---|
| Pauli detector blind to replay/unauthorised | Certain | Medium | Realised & honestly documented (DR 0.600 vs 1.000 statistical); classical rules close the gap |
| Overclaiming real-hardware/CSR applicability | Medium | High | Mitigated — docs explicitly exclude (`docs/limitations.md`) |
| Legal overclaim against AI/ML requirement | Low | High | Mitigated — no AI/ML; documented rationale |
| Reproducibility drift | Low | Medium | Mitigated — fixed seed + typed schema |
| Scope creep into web/API/DB | Medium | Medium | Deferred to FT-049/052/053/054 |

## 7. DoD

Unchanged (all tickets in MVP scope satisfy: code + tests green + docs updated + results bundle regenerated).

## 8. Traceability

1:1 with implementations named above; detailed requirement mapping in `docs/prd-traceability.md`.

## 9. Corrected sprint plan (retrospective)

| SPRINT | PLANNED | RESULT |
|---|---|---|
| Sprint 1 | EP-01 core | Delivered |
| Sprint 2 | EP-02 + EP-03 | Delivered |
| Sprint 3 | EP-04 + EP-07 + CLI | Delivered |
| Sprint 4 | EP-08 + EP-09 + EP-10 | Delivered |
| Stretch | EP-11 + web items | Partial (046/047/050 done; 049/053/054/055 future) |

## 10. Corrected sprint review

All MVP epics accepted in-closing. Remaining backlog is stretch/web/deployment only.

## 11. Corrected CSV export

```
id,title,type,state,Estimate,Priority,Releases
FT-001,Deterministic simulation configuration,feature,Done,1,Must,2026.1.0
FT-002,Bell-state generation and entanglement,feature,Done,2,Must,2026.1.0
FT-003,Quantum teleportation of a message state,feature,Done,2,Must,2026.1.0
FT-004,Pauli correction logic,feature,Done,1,Must,2026.1.0
FT-005,Projective verification (final measurement),feature,Done,2,Must,2026.1.0
FT-006,Simulate forgery (fundamental),feature,Done,2,Must,2026.1.0
FT-007,Simulate impersonation,feature,Done,2,Must,2026.1.0
FT-008,Simulate replay,feature,Done,2,Must,2026.1.0
FT-009,Simulate unauthorised verification,feature,Done,1,Must,2026.1.0
FT-010,Simulate channel manipulation,feature,Done,2,Must,2026.1.0
FT-011,Mismatch statistics (cryptographic),feature,Done,2,Must,2026.1.0
FT-012,Configurable thresholds,feature,Done,1,Must,2026.1.0
FT-013,Outcome classification,feature,Done,2,Must,2026.1.0
FT-014,Explain threat-detection evidence,feature,Done,2,Must,2026.1.0
FT-015,Experiment result schema,feature,Done,2,Must,2026.1.0
FT-016,Save experiment results,feature,Done,2,Must,2026.1.0
FT-017,Export results,feature,Done,1,Must,2026.1.0
FT-018,Validate experiment inputs,feature,Done,1,Must,2026.1.0
FT-019,Protocol execution service,feature,Done (in-process),2,Must,2026.1.0
FT-020,Attack execution service,feature,Done (in-process),2,Must,2026.1.0
FT-021,Detection integration,feature,Done,1,Must,2026.1.0
FT-022,Backend error conventions,feature,Done (CLI-level),1,Should,2026.1.0
FT-023,Experiment configuration screen,feature,Done (CLI),2,Must,2026.1.0
FT-024,QDS protocol workflow view,feature,Done (CLI),2,Must,2026.1.0
FT-025,Measurement-statistics visualisation,feature,Done (console),2,Must,2026.1.0
FT-026,Threat verdict panel,feature,Done (console),2,Must,2026.1.0
FT-027,Loading / empty states,feature,N/A,1,Should,future
FT-028,Actionable frontend errors,feature,Done (CLI),1,Should,2026.1.0
FT-029,Evaluation experiment matrix,feature,Done,2,Must,2026.1.0
FT-030,Benchmark vs classical baseline,feature,Partial,2,Must,future
FT-031,Attack-detection performance,feature,Done,2,Must,2026.1.0
FT-032,Computational performance,feature,Done,1,Should,2026.1.0
FT-033,Benchmark summary view,feature,Done (console),1,Should,2026.1.0
FT-034,Log security-relevant events,feature,Done,2,Must,2026.1.0
FT-035,Sanitise untrusted inputs,feature,Done,1,Must,2026.1.0
FT-036,Handle execution failures,feature,Partial,1,Should,future
FT-037,Protocol unit tests,feature,Done,2,Must,2026.1.0
FT-038,Attack/detector tests,feature,Done,2,Must,2026.1.0
FT-039,End-to-end integration test,feature,Done,2,Must,2026.1.0
FT-040,Reproducible local deployment,feature,Done,2,Must,2026.1.0
FT-041,Offline demo fallback,feature,Done,2,Must,2026.1.0
FT-042,Document the math model,task,Done,2,Must,2026.1.0
FT-043,Document the threat model,task,Done,1,Must,2026.1.0
FT-044,README,task,Done,1,Must,2026.1.0
FT-045,SIH demo script / walkthrough,task,Done,1,Must,2026.1.0
FT-046,Batch experiment runner,feature,Done,3,Could,2026.1.0
FT-047,Configurable noise models,feature,Done,3,Could,2026.1.0
FT-048,Multi-threshold comparison,feature,Partial,3,Could,future
| FT-049,Experiment history dashboard,feature,Partial (read-only viewer),3,Could,future
FT-050,Downloadable security report,feature,Done,2,Could,2026.1.0
FT-051,Real quantum backend,feature,Not Started,13,Could,future
FT-052,PostgreSQL persistence,feature,Not Started,5,Could,future
FT-053,Auth / access control,feature,Not Started,5,Could,future
FT-054,Containerisation / deployment,task,Not Started,3,Could,future
FT-055,CI pipeline,task,Not Started,2,Could,future
```